TCP Congestion Control Across an Emulated WAN

CUBIC, BBR and Reno compared through two VyOS routers, a 100 Mbit/s bottleneck and an adjustable round-trip time — a hands-on lab walkthrough with live measurements

ABSTRACT

This paper documents a reproducible QEMU/libvirt lab in which one Linux host per congestion-control algorithm sends across a netem/tbf WAN emulator. It shows what each algorithm achieves alone, what happens when they share the bottleneck, and how round-trip time, buffer depth, random loss and MTU change the outcome. Every chart, table and screenshot comes from runs on the live lab. Headline results: BBR takes ~86 of 94 Mbit/s against CUBIC and Reno at 30 ms and can starve an established CUBIC flow; 1 % loss caps CUBIC and Reno near 4 Mbit/s while BBR holds ~93; a 4 × BDP buffer lets CUBIC quadruple the path delay; jumbo frames add ~4 % goodput. A controlled single-flow comparison at 500 Mbit/s (iperf3 -t 30 -b 500M) shows where BBR wins on long fat networks: with 0.01 % random loss it carries 13.9× CUBIC's throughput at 150 ms, while on a clean path all three are within 2 %.

Daniel Cantor

October 23, 2026

Version 0.4.0 · 22 of 22 tests passing

libvirt/KVM (QEMU) · VyOS rolling · Alpine 3.22 (Linux 6.12 LTS) · iperf3 3.19

Lab directory `~/tcp-cc-lab` · GitHub `dcantor/tcp-cc-lab` · portal port 8112

Contents

1. What the lab is, and why
2. Congestion control for dummies
3. Reno, CUBIC and BBR in five minutes
4. Architecture
5. How it is built
6. Getting started
7. Walkthrough A — the lab comes up
8. Walkthrough B — each algorithm alone
9. Walkthrough C — all three at once
10. Walkthrough D — the RTT slider
11. Walkthrough E — the buffer: bloat and shallow
12. Walkthrough F — random loss
13. Walkthrough G — moving the RTT under a flow
14. Walkthrough H — late arrivals
15. Walkthrough I — jumbo frames
16. Walkthrough J — 500 Mbit/s, RTT 0 to 150 ms, MTU 1500 / 9000
17. Walkthrough K — is BBR better on long fat networks?
18. The portal
19. Monitoring and the Lab Hub
20. Tests
21. Findings
22. Lessons learned building it
23. Troubleshooting
24. What is not built yet

Appendix A — lab.sh

Appendix B — the WAN emulator script

Appendix C — lab.conf

Appendix D — every TCP setting on the hosts

1. What the lab is, and why

Every TCP sender decides, on its own, how fast to send. That decision — **congestion control** — is made by an algorithm in the sender's kernel, and Linux ships several. Three matter in practice: **Reno**, the classic from 1990 that every textbook describes; **CUBIC**, the Linux default since 2006 and the most common algorithm on the internet; and **BBR**, Google's model-based algorithm from 2016, used for YouTube and much of Google's traffic. They disagree about what congestion *is*, so they behave very differently on the same path — and very differently when they share one.

This lab puts them side by side on a path you control. Two VyOS routers are joined through a WAN emulator whose round-trip time is a slider (default 30 ms, 0 to 300 ms), with a 100 Mbit/s bottleneck, a configurable buffer and optional random loss. Behind each router sit three Linux hosts, one per algorithm. Every flow goes from a host in site A to the host with the same algorithm in site B, across the same bottleneck.

> **What this lab is for.** Behaviour, not benchmark numbers. The VMs share one 12-core machine, so absolute numbers are not those of a real WAN — but the bottleneck is real (a token bucket at 100 Mbit/s), the queue is real (its size in bytes is set and counted), the delay is real (netem holds each packet for half the RTT each way), and the senders are unmodified Linux kernels running their stock algorithms. Who wins, by how much, and at what cost in queueing delay is exactly what the lab can show.

- **QEMU/KVM, like the SRv6 labs.** Nine libvirt VMs on isolated libvirt bridges; no containers. The routers are real VyOS, configured on first boot over their serial consoles.
- **One knob at a time.** RTT, rate, loss, buffer and the hosts' MTU each change independently, in place, from the portal or the command line — the RTT even under a running flow.
- **Measured from the senders.** iperf3's per-second JSON gives each flow's throughput, smoothed RTT, congestion window and retransmits as they happen; the emulator's queue counters give the drops. Robot Framework suites turn the known behaviours into checks.

2. Congestion control for dummies

No networking background needed. If you know what a congestion window is, skip to section 3.

The problem: nobody tells a sender how fast it may go

When your computer downloads a file, the data crosses a chain of links — Wi-Fi, a home router, the ISP, long-distance fibre, a data centre. The slowest link on the path, the **bottleneck**, decides how fast the transfer can go. But no one tells the sender which link that is, how fast it is, or how many other people are using it right now. The sender must find out by trying.

Send too slowly and the link sits idle. Send too fast and packets pile up in the bottleneck's **buffer** (a waiting line in the router), and when the buffer is full, packets are thrown away and must be sent again. Congestion control is the sender's strategy for finding the right speed — and for sharing it fairly with everyone else doing the same thing.

Three words you need

1. **Round-trip time (RTT).** How long a packet takes to get to the other end and the acknowledgement to come back. Across a city, a few milliseconds; across a continent, 30 to 80 ms; across an ocean, 100 to 300 ms. The sender only learns what happened to a packet one RTT later.
2. **The window.** How much data the sender lets be "in flight" — sent but not yet acknowledged. Speed is window divided by RTT: to go 100 Mbit/s over a 30 ms path, about 375 KB must be in flight at all times. That amount is the **bandwidth-delay product (BDP)**, the size of the "pipe".
3. **The buffer.** The bottleneck router's waiting line. A packet that arrives while the link is busy waits there. A full buffer is a long wait (delay goes up); an overflowing buffer is a drop (a packet is lost). This lab sizes the buffer in BDPs: 1 × BDP at 30 ms holds 30 ms of data.

> **An analogy.** The bottleneck is a single toll booth; the buffer is the queue of cars in front of it; the RTT is the time it takes a driver to hear back from the booth. **Reno** and **CUBIC** drivers keep pushing more cars into the queue until one is turned away, then send half as many — so the queue is always nearly full and everyone waits. **BBR** drivers watch how fast cars actually get through and how long the trip takes when the queue is empty, and send just that many — so the queue stays short. Put them on the same road and the polite strategy and the pushy strategy do not share evenly: who wins depends on how long the queue is allowed to get.

Why it matters

- **Throughput on long paths.** The longer the RTT, the longer a loss-based sender takes to grow its window back after a drop. Reno on a transatlantic path can take minutes to recover from one loss; CUBIC was designed to fix exactly that.
- **Delay for everyone else.** A sender that fills the buffer adds that buffer's delay to every other packet crossing the link: the video call stutters because someone is uploading a backup. This is **bufferbloat**.
- **Fairness.** When different algorithms share a bottleneck, one can starve the others. Every network operator who lets the public on their links lives with the mix.
- **Loss that isn't congestion.** On Wi-Fi or a lossy radio link, packets get lost even when the link is empty. Reno and CUBIC take every loss as "slow down"; BBR does not.

The three, side by side

	Reno	CUBIC	BBR (v1, as in Linux 6.12)
what it reacts to	packet loss	packet loss	measured bandwidth and minimum RTT
after a loss	halves the window	cuts it to 70 %	nothing special (loss is not a signal)
growing again	+1 packet per RTT — slow on long paths	a cubic curve in *time*, independent of RTT	paces at the estimated bottleneck rate, probes 25 % above it every 8 RTTs
the bottleneck queue	fills it	fills it	keeps it small (~1 BDP in flight + a little)
sharing with the others	fair with itself, loses to CUBIC	fair with itself, beats Reno	takes the most when buffers are shallow; less when they are deep
in Linux	always	default since 2.6.19 (2006)	since 4.9 (2016), opt-in per host or socket

3. Reno, CUBIC and BBR in five minutes

The window, slow start and AIMD

A TCP sender keeps a **congestion window** (cwnd): how many bytes it may have unacknowledged. Throughput ≈ cwnd / RTT. A new connection starts with 10 segments and doubles the window every RTT (**slow start**) until a loss — or until it reaches `ssthresh`, the level where the last loss happened. After that it is in **congestion avoidance**. Reno's congestion avoidance is **AIMD**: *additive increase* of one segment per RTT, *multiplicative decrease* by half on a loss. On a 100 Mbit/s, 30 ms path with 1500-byte packets the pipe is ~250 segments; after a halving, Reno needs 125 RTTs — almost 4 s — to fill it again. At 160 ms, 1330 segments and 665 RTTs: almost two minutes.

CUBIC

CUBIC (Ha, Rhee, Xu, 2008; RFC 9438) keeps the loss response but replaces the linear growth with a cubic function of the **time since the last loss**: the window grows fast at first, flattens out as it approaches the size it had when the loss happened (W_{max}), lingers there, and then accelerates again to probe for more. Because growth depends on time and not on RTTs, a long-RTT CUBIC flow recovers about as fast in seconds as a short-RTT one. It multiplies the window by 0.7 on a loss (Reno: 0.5). It still fills the buffer: it only backs off when the buffer overflows.

BBR

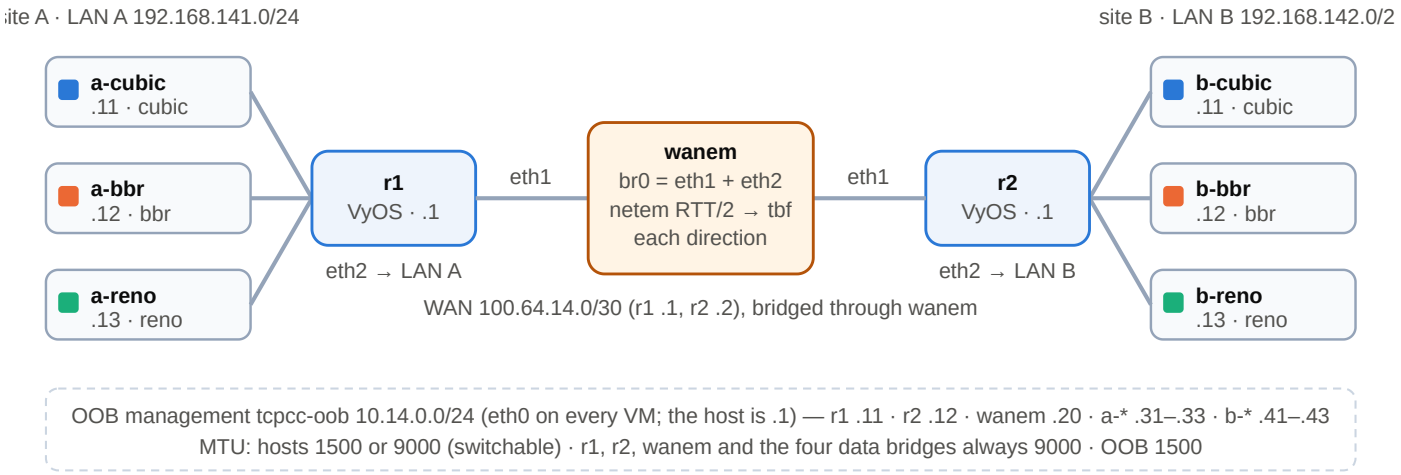
BBR (Cardwell et al., Google, 2016) — "Bottleneck Bandwidth and Round-trip propagation time" — does not use loss as its signal. It continuously estimates two numbers: **BtlBw**, the maximum delivery rate seen over the last ~10 RTTs, and **RTprop**, the minimum RTT seen over the last 10 s. Their product is the BDP. BBR *paces* its packets (this lab puts the `fq` qdisc on every host for that) at BtlBw, keeps about 2 × BDP in flight, and cycles its pacing gain through 1.25, 0.75, 1, 1, 1, 1, 1, 1 — one RTT above the estimate to look for more bandwidth, one below to drain whatever queue that built. Every 10 s it drops to 4 packets in flight for 200 ms (**ProbeRTT**) to re-measure the empty-pipe RTT.

> **The known catch, which this lab reproduces.** BBR v1 ignores loss, so in a shallow buffer it keeps sending at its estimate while the loss-based flows next to it see drops and back off — BBR takes most of the link. In a deep buffer the loss-based flows fill the queue, BBR's RTT samples go up, and its in-flight cap (2 × its BDP estimate) lets CUBIC claim more. BBRv2/v3 add a loss and ECN response to fix the first case; they are not in mainline Linux 6.12, which this lab runs.

What the emulator changes, and what it means

setting	range	effect on the algorithms
RTT	0–300 ms (default 30)	a bigger pipe (BDP) to fill; slower feedback; Reno's recovery time grows with RTT^2, CUBIC's does not
rate	1–1000 Mbit/s (default 100)	the bottleneck; also scales the BDP
loss	0–20 % random, each way (default 0)	non-congestion loss: Reno and CUBIC back off on every drop, BBR does not
buffer	0.05–16 × BDP (default 1)	shallow: loss-based flows lose often; deep: they fill it and the RTT grows by up to the buffer's drain time
host MTU	1500 or 9000	six times fewer, bigger segments: a window grows six times faster in bytes per RTT

4. Architecture



Every flow runs from a site A host to its same-algorithm peer in site B (or back), through r1, wanem's bottleneck and r2.

Nodes

node	what	vCPU	RAM	data	OOB
r1, r2	VyOS (overlay on the shared `vyos-base.qcow2`), static routes for the far LAN	2	1 GiB	eth1 WAN, eth2 LAN	.11, .12
wanem	Alpine: Linux bridge br0 between eth1 (to r1) and eth2 (to r2), netem + tbf on each egress	2	512 MiB	eth1, eth2 (no IP)	.20
a-cubic, a-bbr, a-reno	Alpine hosts in site A, one algorithm each, iperf3 server on 5201	1	256 MiB	192.168.141.11–13	.31–.33
b-cubic, b-bbr, b-reno	the same in site B; each host's peer is the same algorithm on the other side	1	256 MiB	192.168.142.11–13	.41–.43

Libvirt domains are `tcpcc-<node>`, serial consoles on 127.0.0.1:5601–5609, MACs `52:54:00:ce:<node>:<nic>`. Users: routers vyos / vyos, the rest lab / lab; your SSH key is installed everywhere. About 3.8 GiB of RAM in all.

The links: bridges, not tunnels

The five networks are isolated libvirt bridges: `tcpcc-oob` (with the host as gateway) and four data segments — `tcpcc-wan-a` (r1 eth1 ↔ wanem eth1), `tcpcc-wan-b` (wanem eth2 ↔ r2 eth1), `tcpcc-lan-a`, `tcpcc-lan-b`. The SRv6 labs use QEMU UDP socket tunnels for point-to-point links; this lab doesn't, because the UDP backend keeps the host's default 208 KB socket buffer and silently drops bursts (found in homa-lab). A loss nobody can see would bias exactly what this lab measures.

The WAN emulator

wanem is a transparent bridge: r1 and r2 see each other on the /30 as if on one wire. Each egress port carries the same two-stage impairment, so the two directions are independent (the data direction and the ACK direction each get half the RTT):

```
root 1:   netem  delay RTT/2, loss L % (random), limit = 2 × packets in flight on the delay line + 1000
  └ 10:   tbf    rate R, burst max(R / 250 Hz, 2 jumbo frames), limit = BUFFER × BDP bytes (at least 4 jumbo
```

```
frames)
```

- **Delay first, then the bottleneck.** The queue a congestion control fills, and where it loses packets, is tbf's. Its size in bytes is set directly from the BDP and does not move when the delay does.
- **Changed in place.** `wanem apply` uses `tc qdisc change`, not replace: a running flow sees a new RTT within one packet, with no reset of the queue.
- **Wire-size packets.** Offloads are off on wanem (GRO, LRO, TSO, GSO), or netem and tbf would see 64 KB super-packets and a queue limit would mean nothing. The bytes-per-packet through tbf (≈1512 at MTU 1500, ≈9000 at 9000) confirms it.
- **netem's own limit** defaults to 1000 packets — too few for a long fat pipe (100 Mbit/s × 150 ms one way = 1250 packets on the delay line). It would drop like a random-loss link, so wanem sizes it from the RTT and rate on every change.

The hosts

- `net.ipv4.tcp_congestion_control` = the host's algorithm; `fq` on eth1 (BBR paces through it; CUBIC and Reno get the same qdisc so the comparison is fair).
- 32 MB maximum socket buffers: at 300 ms × 100 Mbit/s the BDP is 3.75 MB; with the Linux defaults (4–6 MB) a window cap, not the algorithm, would limit throughput.
- `tcp_no_metrics_save = 1`: otherwise Linux caches the last connection's ssthresh per destination and the next run starts where the previous one ended.
- An iperf3 server on 5201; the flows are `iperf3 -c <peer> --json-stream` started over SSH, parsed second by second.

The hosts' TCP settings

Alpine 3.22's kernel defaults plus seven lines the lab sets from cloud-init. The table lists the settings that shape a bulk TCP flow, read live from the six hosts; Appendix D lists every `net.ipv4.tcp_*` and `net.core` buffer value.

setting	value	set by	what it does
tcp_congestion_control	cubic · bbr · reno	lab	the algorithm: one per host pair, the only setting that differs between the six
tcp_available_congestion_control	reno cubic bbr	kernel	modules loaded (tcp_bbr by the lab; Reno is built in)
core.default_qdisc	fq	lab	qdisc for new interfaces; eth1 also gets fq explicitly — fq does the pacing BBR relies on
tcp_rmem	4096 131072 33554432	lab	receive buffer min / default / max, bytes; autotuning grows a socket up to max
tcp_wmem	4096 65536 33554432	lab	send buffer min / default / max, bytes
core.rmem_max	33554432	lab	cap for SO_RCVBUF set by an application
core.wmem_max	33554432	lab	cap for SO_SNDBUF set by an application
tcp_mem	2418 3227 4836 = 9.9 / 13.2 / 19.8 MB	kernel	all TCP sockets together, in 4 KiB pages: low / pressure / high; sized by the kernel from RAM
tcp_moderate_rcvbuf	1	kernel	receive-buffer autotuning on
tcp_adv_win_scale	1	kernel	share of the receive buffer advertised as window (1 = half)
tcp_window_scaling	1	kernel	RFC 7323 window scaling: windows above 64 KB
tcp_timestamps	1	kernel	RFC 7323 timestamps: RTT samples on every ACK, PAWS
tcp_sack	1	kernel	selective ACKs
tcp_dsack	1	kernel	duplicate SACKs (detects spurious retransmits)
tcp_recovery	1	kernel	1 = RACK-TLP loss detection (time-based, not 3 dup-ACKs)
tcp_early_retrans	3	kernel	3 = tail loss probe on
tcp_frto	2	kernel	F-RTO: detects spurious retransmission timeouts
tcp_reordering	3	kernel	initial reordering tolerance, segments
tcp_ecn	2	kernel	2 = accept ECN when the peer asks, do not ask (the WAN does not mark anyway)
tcp_no_metrics_save	1	lab	1 = forget cwnd / ssthresh / RTT of a finished connection, so each run starts clean
tcp_slow_start_after_idle	1	kernel	1 = shrink cwnd after an idle period (irrelevant to bulk iperf3 flows)
tcp_pacing_ss_ratio	200	kernel	pacing rate in slow start, % of cwnd/RTT
tcp_pacing_ca_ratio	120	kernel	pacing rate in congestion avoidance, % of cwnd/RTT
tcp_limit_output_bytes	1048576	kernel	TSQ: bytes a socket may have queued below TCP (qdisc + NIC)
tcp_notsent_lowat	4294967295	kernel	unsent bytes allowed in the send buffer (unlimited)
tcp_rto_min_us	200000	kernel	minimum retransmission timeout, µs
tcp_min_rtt_wlen	300	kernel	window for the min-RTT filter, s
tcp_mtu_probing	0	kernel	0 = no packetisation-layer PMTU discovery (the path MTU is known)
tcp_fastopen	1	kernel	1 = TCP Fast Open as a client
core.netdev_max_backlog	1000	kernel	receive backlog per CPU, packets

eth1 MTU	1500	lab	1500 by default; 9000 with lab.sh mtu set 9000 (section 15)
eth1 offloads	TSO on, GSO on, GRO on	kernel	virtio-net segmentation and receive offloads: TCP hands the NIC up to 64 KB at a time; wanem has them off so tbf sees real packets

Read from all six hosts (a-cubic, a-bbr, a-reno, b-cubic, b-bbr, b-reno) on October 23, 2026, kernel 6.12.81-0-virt. The values are identical on all six except the algorithm. “lab”: set by /etc/sysctl.d/90-tcpcc.conf or /etc/local.d/tcpcc.start from cloud-init; “kernel”: Alpine’s default.

What the lab sets, `/etc/sysctl.d/90-tcpcc.conf` (a-cubic's; the other hosts differ only in the first line), applied with the MTU, fq and the iperf3 server by `/etc/local.d/tcpcc.start` at boot:

```
# /etc/sysctl.d/90-tcpcc.conf
net.ipv4.tcp_congestion_control = cubic
net.core.default_qdisc = fq
net.core.rmem_max = 33554432
net.core.wmem_max = 33554432
net.ipv4.tcp_rmem = 4096 131072 33554432
net.ipv4.tcp_wmem = 4096 65536 33554432
net.ipv4.tcp_no_metrics_save = 1

# /etc/local.d/tcpcc.start
#!/bin/sh
# tcp-cc-lab host: the algorithm and buffers (sysctl), the MTU (lab.sh mtu), fq on the data NIC, the iperf3
server
modprobe tcp_bbr; modprobe sch_fq
sysctl -q -p /etc/sysctl.d/90-tcpcc.conf
ip link set eth1 mtu "$(cat /etc/tcpcc-mtu 2>/dev/null || echo 1500)"
tc qdisc replace dev eth1 root fq
pgrep -x iperf3 >/dev/null || iperf3 -s -D -p 5201 --logfile /var/log/iperf3.log
```

> **tcp_mem is small on these hosts.** The kernel sizes it from RAM, and the hosts have 217 MB: TCP as a whole enters memory pressure at 13.2 MB and stops growing buffers at 19.8 MB. That is plenty at 100 Mbit/s (BDP 3.75 MB at 300 ms), but not at 500 Mbit/s on long paths: one CUBIC flow alone at 150 ms (BDP 9.4 MB) put the sender into pressure 145 times and the receiver 0 times in 30 s (`TcpExt TCPMemoryPressures`) and got 438 Mbit/s. Under pressure the kernel stops growing socket buffers, so the 100 and 150 ms points of section 16 may be partly limited by the hosts, not only by the algorithm. Raising the hosts' RAM or `net.ipv4.tcp_mem` would remove the limit.

MTU

r1 and r2's eth1/eth2, wanem's eth1/eth2/br0 and the four data bridges are MTU 9000, always. The hosts' eth1 is 1500 by default and switches to 9000 and back, all six together. So the infrastructure never fragments or drops anything a host sends, and the only thing the switch changes is the segment size TCP negotiates (MSS 1448 or 8948). The OOB network stays at 1500.

5. How it is built

1. **The Alpine image** (`./lab.sh image`, ~2 min). A throw-away builder VM boots the Alpine 3.22 NoCloud image on libvirt's NAT network; cloud-init installs iperf3, iproute2 (full `tc` and `ss`), ethtool, tcpdump, mtr, curl and iputils ping (busybox ping can't set don't-fragment), checks the kernel has BBR, netem, tbf and fq, cleans itself and powers off. Its disk becomes `images/tcpcc-alpine.qcow2`; the build fails if `CHECK cc:` doesn't list bbr.
2. **The VMs** (`./lab.sh up`). Each node gets a qcow2 overlay on its base image and a NoCloud seed ISO rendered from `lab.conf`: network config by MAC, `/etc/hosts`, doas (Alpine has no sudo), and a `local.d` start script — the hosts' sysctls, MTU, fq and iperf3; wanem's bridge and `wanem reset`.
3. **The routers' day 0.** VyOS has no cloud-init seed here, so the first boot of r1 and r2 pushes their configuration over the serial console (`tools/vyos_console.py`, from srv6-core): log in, `configure`, the set commands, `commit`, `save`. A marker file stops it happening again; `./lab.sh configure` re-applies it over SSH later.
4. **Verify** (`./lab.sh verify`): every host's algorithm, the WAN settings, the MTU of every infrastructure link, and a ping between every A/B pair — at the full host MTU with don't-fragment set.

The host part of the seed (rendered per host; the algorithm is the only line that differs):

```
  - path: /etc/sysctl.d/90-tcpcc.conf
    content: |
      net.ipv4.tcp_congestion_control = ${CC[$n]}
$(for kv in "${HOST_SYSCTL[@]}"; do echo "      ${kv%%=*} = ${kv#*=}"; done)
  - path: /etc/local.d/tcpcc.start
    permissions: "0755"
    content: |
      #!/bin/sh
      # tcp-cc-lab host: the algorithm and buffers (sysctl), the MTU (lab.sh mtu), fq on the data NIC, the
iperf3 server
      modprobe tcp_bbr; modprobe sch_fq
      sysctl -q -p /etc/sysctl.d/90-tcpcc.conf
      ip link set eth1 mtu "\$(cat /etc/tcpcc-mtu 2>/dev/null || echo $HOST_MTU)"
      tc qdisc replace dev eth1 root fq
      pgrep -x iperf3 >/dev/null || iperf3 -s -D -p 5201 --logfile /var/log/iperf3.log
```

A router's configuration (r1; r2 mirrors it):

```
set interfaces ethernet eth1 address '100.64.14.1/30'
set interfaces ethernet eth1 description 'WAN to r2 via wanem'
set interfaces ethernet eth1 mtu '9000'
set interfaces ethernet eth2 address '192.168.141.1/24'
set interfaces ethernet eth2 description 'LAN site a'
set interfaces ethernet eth2 mtu '9000'
set protocols static route 10.0.0.0/8 next-hop 10.14.0.1
set protocols static route 192.168.142.0/24 next-hop 100.64.14.2
set system host-name 'r1'
```

6. Getting started

```
cd ~/tcp-cc-lab
./lab.sh image                        # once: images/tcpcc-alpine.qcow2 (~2 min)
./lab.sh up                           # first boot also pushes the routers' day-0 config (~3 min)
```

```
./lab.sh verify                        # algorithms, WAN settings, MTU, routes, RTT between every A/B pair
./lab.sh wan apply 100 100 0 1         # RTT 100 ms, 100 Mbit/s, 0 % loss, buffer 1 × BDP
./lab.sh flows compete --duration 30   # all three at once through the bottleneck
./lab.sh flows solo --algos cubic,bbr  # one at a time
./lab.sh mtu set 9000                  # hosts to jumbo (and back with 1500)
./lab.sh test                          # the Robot suites → results/latest
```

Or from the **portal** (section 18) at `http://192.168.50.231:8112`, behind the Lab Hub sign-in, or from the **Lab Hub** card, which starts, saves and resumes the whole lab.

7. Walkthrough A — the lab comes up

`./lab.sh status` lists the nodes from `lab.conf` and libvirt's view of them:

```
NODE     ROLE    STATE      MGMT-IP     DATA-IP         CC     CONSOLE  VM
r1       router  running    10.14.0.11  192.168.141.1   -      5601     tcpcc-r1
r2       router  running    10.14.0.12  192.168.142.1   -      5602     tcpcc-r2
wanem    wanem   running    10.14.0.20  bridge          -      5603     tcpcc-wanem
a-cubic  host    running    10.14.0.31  192.168.141.11  cubic  5604     tcpcc-a-cubic
a-bbr    host    running    10.14.0.32  192.168.141.12  bbr    5605     tcpcc-a-bbr
a-reno   host    running    10.14.0.33  192.168.141.13  reno   5606     tcpcc-a-reno
b-cubic  host    running    10.14.0.41  192.168.142.11  cubic  5607     tcpcc-b-cubic
b-bbr    host    running    10.14.0.42  192.168.142.12  bbr    5608     tcpcc-b-bbr
b-reno   host    running    10.14.0.43  192.168.142.13  reno   5609     tcpcc-b-reno

WAN: r1 100.64.14.1 <-> wanem (br0) <-> r2 100.64.14.2; defaults RTT 30 ms, 100 Mbit/s, loss 0 %, buffer 1 x
BDP
```

`./lab.sh verify` — each host's algorithm and qdisc, the emulator's settings, the jumbo links, and a don't-fragment ping between every pair. Nine pairs × ~31 ms: the emulated 30 ms plus about 1 ms of the VMs' own.

```
== hosts
a-cubic  cc cubic  (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
a-bbr    cc bbr    (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
a-reno   cc reno   (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-cubic  cc cubic  (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-bbr    cc bbr    (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-reno   cc reno   (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up

== wanem
RTT_MS=30
RATE_MBIT=100
LOSS_PCT=0
BUFFER_BDP=1
ONE_WAY_US=15000
QUEUE_BYTES=375000
BDP_BYTES=375000
APPLIED=1792729933

== routers
  [r1] eth1             UP             100.64.14.1/30 fe80::5054:ff:fece:101/64
  [r1] eth2             UP             192.168.141.1/24 fe80::5054:ff:fece:102/64
  [r1] 192.168.141.0/24 dev eth2 proto kernel scope link src 192.168.141.1
  [r1] 192.168.142.0/24 nhid 18 via 100.64.14.2 dev eth1 proto static metric 20
  [r2] eth1             UP             100.64.14.2/30 fe80::5054:ff:fece:201/64
  [r2] eth2             UP             192.168.142.1/24 fe80::5054:ff:fece:202/64
  [r2] 192.168.141.0/24 nhid 18 via 100.64.14.1 dev eth1 proto static metric 20
  [r2] 192.168.142.0/24 dev eth2 proto kernel scope link src 192.168.142.1

== infrastructure MTU (must be 9000)
  r1, r2, wanem: every data link 9000

== across the WAN: every site A host to every site B host (ping, 5 x 0.2 s; one unfragmentable at the hosts'
MTU)
  a-cubic  -> b-cubic  rtt avg 30.811 ms
  a-cubic  -> b-bbr    rtt avg 30.885 ms
```

```
  a-cubic  -> b-reno   rtt avg 30.838 ms
  a-bbr    -> b-cubic  rtt avg 30.819 ms
  a-bbr    -> b-bbr    rtt avg 30.851 ms
  a-bbr    -> b-reno   rtt avg 30.889 ms
  a-reno   -> b-cubic  rtt avg 30.823 ms
  a-reno   -> b-bbr    rtt avg 30.838 ms
  a-reno   -> b-reno   rtt avg 30.815 ms
  9/9 pairs ok (hosts' MTU 1500: 1472-byte pings, DF set)
```

`./lab.sh cc` asks each host which algorithm its kernel uses, which are available, the MTU, the qdisc and whether iperf3 is listening:

```
a-cubic  cc cubic  (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
a-bbr    cc bbr    (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
a-reno   cc reno   (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-cubic  cc cubic  (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-bbr    cc bbr    (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
b-reno   cc reno   (available: reno cubic bbr)  mtu 1500  qdisc eth1: fq  iperf3: up
```

On a-bbr, the settings behind that line:

```
net.ipv4.tcp_congestion_control = bbr
net.ipv4.tcp_available_congestion_control = reno cubic bbr
net.core.default_qdisc = fq
net.ipv4.tcp_no_metrics_save = 1
net.ipv4.tcp_rmem = 4096        131072  33554432
net.ipv4.tcp_wmem = 4096        65536   33554432
net.core.rmem_max = 33554432
net.core.wmem_max = 33554432
qdisc fq 8001: root refcnt 2 limit 10000p flow_limit 100p buckets 1024 orphan_mask 1023 bands 3 priomap 1 2 2
2 1 2 0 0 1 1 1 1 1 1 1 1 weights 589824 196608 65536 quantum 3028b initial_quantum 15140b low_rate_threshold
550Kbit refill_delay 40ms timer_slack 10us horizon 10s horizon_drop
```

r1's routing table: the far LAN via r2 across the WAN /30, management via the host.

```
S>* 10.0.0.0/8 [1/0] via 10.14.0.1, eth0, weight 1, 01:32:03
C>* 10.14.0.0/24 is directly connected, eth0, weight 1, 01:32:04
L>* 10.14.0.11/32 is directly connected, eth0, weight 1, 01:32:04
C>* 100.64.14.0/30 is directly connected, eth1, weight 1, 01:31:53
L>* 100.64.14.1/32 is directly connected, eth1, weight 1, 01:31:53
C>* 192.168.141.0/24 is directly connected, eth2, weight 1, 01:31:54
L>* 192.168.141.1/32 is directly connected, eth2, weight 1, 01:31:54
S>* 192.168.142.0/24 [1/0] via 100.64.14.2, eth1, weight 1, 01:32:03
```

wanem's qdiscs at the defaults (`./lab.sh wan show`). 15 ms each way, a 100 Mbit/s token bucket, and a queue of 375 000 bytes — exactly 1 × BDP (100 Mbit/s × 30 ms). The counters are cumulative since boot.

```
RTT_MS=30
RATE_MBIT=100
LOSS_PCT=0
BUFFER_BDP=1
ONE_WAY_US=15000
QUEUE_BYTES=375000
BDP_BYTES=375000
APPLIED=1792729933
--- eth1 (egress towards r1)
qdisc netem 1: root refcnt 2 limit 1250 delay 15ms seed 17046640348788336126
 Sent 315808004 bytes 1131418 pkt (dropped 2430, overlimits 0 requeues 0)
```

```
 backlog 0b 0p requeues 0
qdisc tbf 10: parent 1:1 rate 100Mbit burst 50000b lat 26ms
 Sent 315808004 bytes 1131418 pkt (dropped 2414, overlimits 1065125 requeues 0)
 backlog 0b 0p requeues 0
--- eth2 (egress towards r2)
qdisc netem 1: root refcnt 2 limit 1250 delay 15ms seed 19708122041511612
 Sent 3822336636 bytes 2066958 pkt (dropped 17914, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc tbf 10: parent 1:1 rate 100Mbit burst 50000b lat 26ms
 Sent 3822336636 bytes 2066958 pkt (dropped 17888, overlimits 11140795 requeues 0)
 backlog 0b 0p requeues 0
```

Offloads, so the qdiscs see real packets, and the bridge:

```
tcp-segmentation-offload: off
generic-receive-offload: off
large-receive-offload: off [fixed]
tcp-segmentation-offload: off
generic-receive-offload: off
large-receive-offload: off [fixed]

eth1             UP             52:54:00:ce:03:01 <BROADCAST,MULTICAST,UP,LOWER_UP>
eth2             UP             52:54:00:ce:03:02 <BROADCAST,MULTICAST,UP,LOWER_UP>
br0              UP             52:54:00:ce:03:01 <BROADCAST,MULTICAST,UP,LOWER_UP>
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 master br0 state forwarding priority 32 cost 100
4: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 master br0 state forwarding priority 32 cost 100
```

The MTU of every link (`./lab.sh mtu show`):

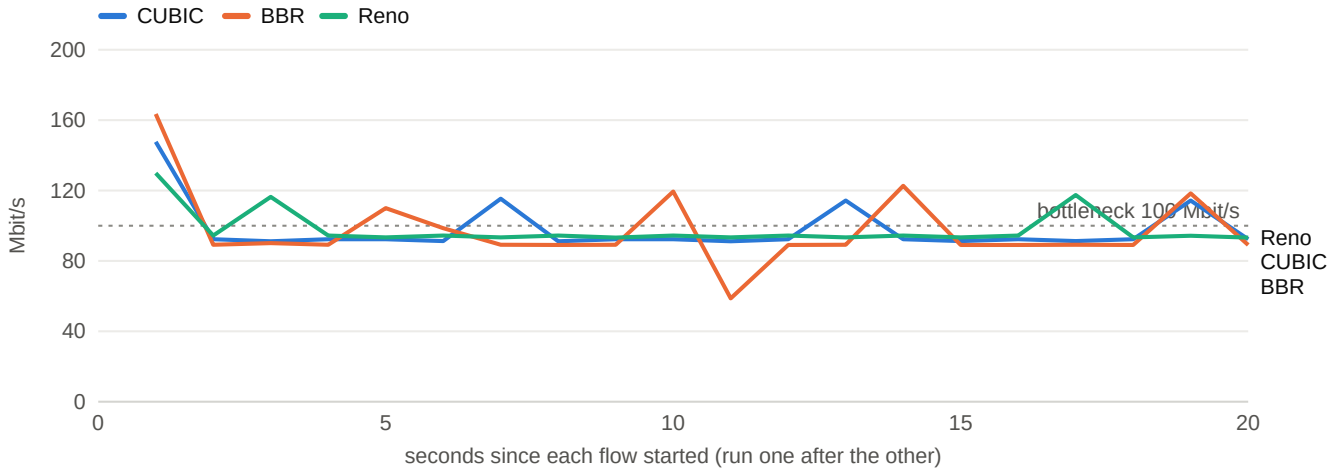
```
r1       eth1 9000  eth2 9000
r2       eth1 9000  eth2 9000
wanem    eth1 9000  eth2 9000  br0 9000
a-cubic  eth1 1500
a-bbr    eth1 1500
a-reno   eth1 1500
b-cubic  eth1 1500
b-bbr    eth1 1500
b-reno   eth1 1500
```

8. Walkthrough B — each algorithm alone

The baseline: each algorithm gets the link to itself for 20 s, one after the other, at the defaults (30 ms, 100 Mbit/s, buffer 1 × BDP, MTU 1500).

```
./lab.sh flows solo --duration 20
```

Throughput, each flow alone (per second)



algorithm (alone)	Mbit/s	retransmits	bottleneck drops	RTT min	RTT mean	RTT max (ms)	MSS	bytes / packet
CUBIC	94.9	499	499	43.5	51.4	59.7	1448	1514
BBR	93.7	1474	1474	34.7	51.1	60.5	1448	1514
Reno	94.8	511	511	30.5	44.4	58.5	1448	1514

Alone, all three fill the link: 94.92, 93.73 and 94.85 Mbit/s of receiver goodput — the ceiling is ~95.6 Mbit/s, because 1448 bytes of payload ride in 1514 bytes on the wire (the bottleneck counted 1514 bytes per packet). The differences are in *how*: BBR retransmitted the most (1474) — its startup overshoots and it keeps up to 2 × BDP in flight, which a 1 × BDP buffer cannot hold. Reno's sawtooth leaves the queue emptier on average (mean RTT 44.37 ms against CUBIC's 51.41 ms). The per-second points are iperf3's *sender-side* counts: they swing above 100 Mbit/s when a second's count catches bytes still in the host's socket buffer. The tables use the receiver's total, which is the real throughput.

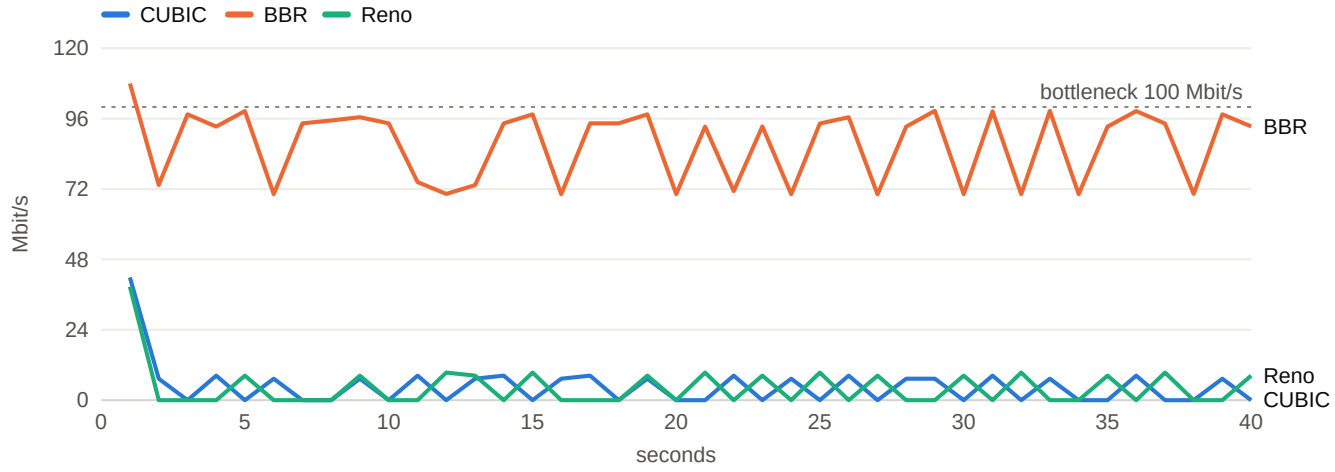
9. Walkthrough C — all three at once

Now the three start together and share the bottleneck for 40 s, same settings.

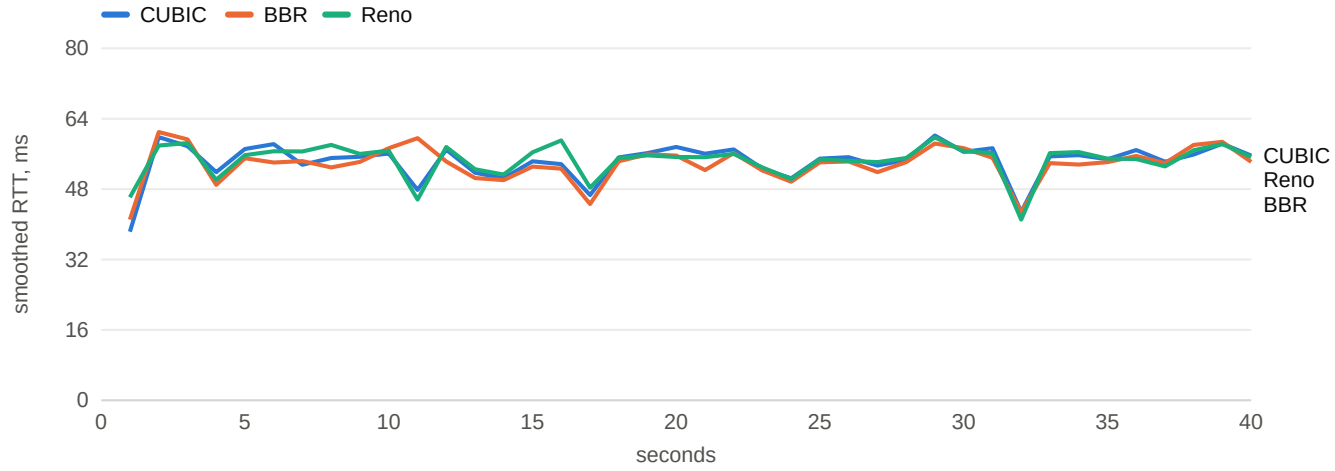
```
./lab.sh flows compete --duration 40
```

> **Jain's fairness index** is $(\Sigma x)^2 / (n \cdot \Sigma x^2)$: 1 when every flow gets the same, 1/n when one flow gets everything (0.33 for three flows).

Throughput, all three sharing 100 Mbit/s, 30 ms, buffer 1 × BDP



Smoothed RTT as each sender sees it



algorithm	flow	Mbit/s	retransmits	RTT min	RTT mean	RTT max (ms)	MSS
CUBIC	`a-cubic → b-cubic`	4.3	368	38.3	54.3	60.2	1448
BBR	`a-bbr → b-bbr`	85.9	4026	41.1	53.7	61.0	1448
Reno	`a-reno → b-reno`	3.7	372	41.1	54.5	59.8	1448
total · Jain fairness 0.396		**93.8**	**4766**	**bottleneck drops 4766**			

Together, the picture is entirely different: BBR 85.92 Mbit/s, CUBIC 4.27, Reno 3.66. All three see the same RTT — about 54 ms, 30 ms of path and ~24 ms of a full queue — because they share one queue. BBR, which does not slow down on

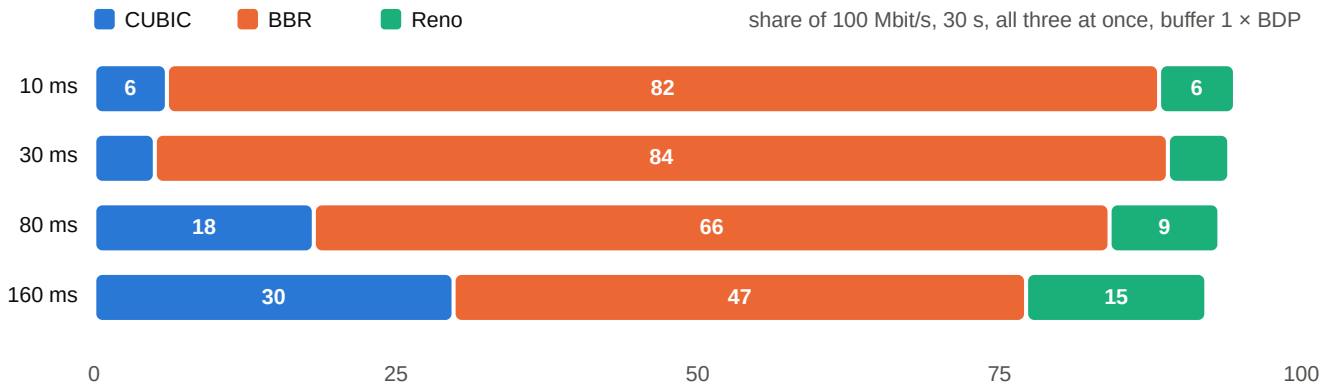
loss and paces at its bandwidth estimate, keeps the queue full and overflowing (4766 drops in 40 s); CUBIC and Reno take every one of those drops as a signal and keep cutting their windows. This is BBR v1's best-known flaw, and the reason BBRv2/v3 added a loss response.

10. Walkthrough D — the RTT slider

The same three-way contest at four round-trip times, 30 s each. The buffer stays 1 × BDP, so it grows with the RTT (it is always "one RTT of data").

```
for rtt in 10 30 80 160; do ./lab.sh wan apply $rtt 100 0 1; ./lab.sh flows compete --duration 30; done
```

Who gets the bottleneck, by RTT (Mbit/s)



RTT (ms)	buffer (1 × BDP)	CUBIC	BBR	Reno (Mbit/s)	total	Jain	drops
10	125 KB	6.0	82.2	6.3	94.5	0.436	6752
30	375 KB	5.0	83.9	5.1	94.0	0.415	3464
80	1000 KB	18.2	65.9	9.1	93.2	0.609	1795
160	2000 KB	29.8	47.4	14.9	92.1	0.842	2614

The longer the path, the more CUBIC claws back: from 6.01 Mbit/s at 10 ms to 29.76 at 160 ms, while BBR falls from 82.19 to 47.43 and fairness rises from 0.436 to 0.842. CUBIC's window grows as a function of *time*, so on a long path it reclaims more of the pipe between losses per RTT than Reno, whose one-segment-per-RTT growth stays about half of CUBIC's at 80 and 160 ms. Why BBR's share shrinks with the RTT is not isolated by this run (the buffer is 1 × BDP at every RTT, so it is not the buffer depth); BBR's drops fall with the RTT too, which suggests it spends less of each loss episode overrunning the queue. Note the total stays at 92–94 Mbit/s throughout: the link is always full, only the split changes.

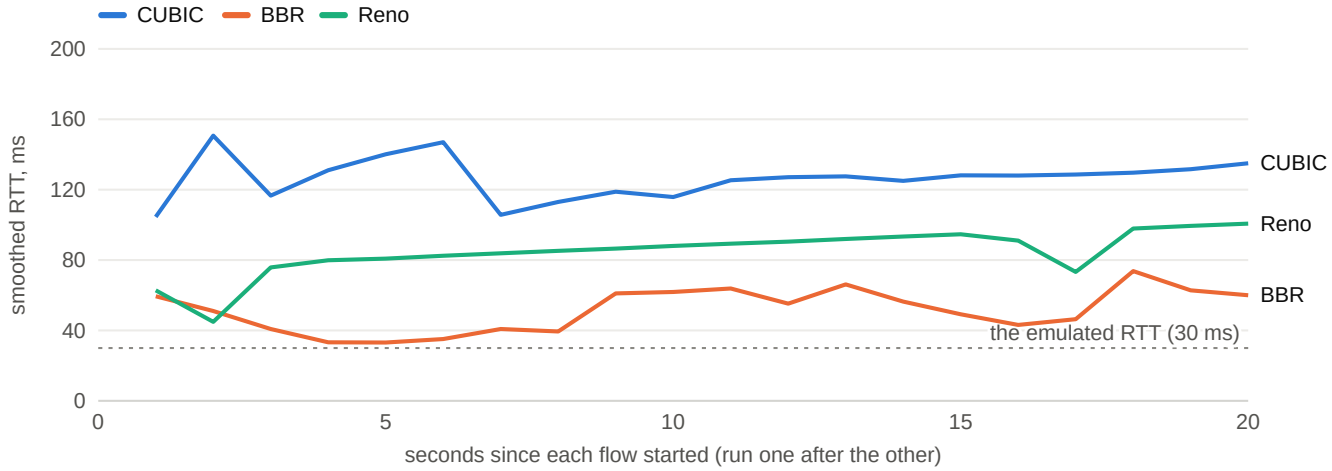
11. Walkthrough E — the buffer: bloat and shallow

A deep buffer: 4 × BDP at 30 ms, each algorithm alone

A buffer of 4 × BDP holds 120 ms of data at 100 Mbit/s — the bufferbloat case: an edge router or a cable modem with far more memory than the link needs.

```
./lab.sh wan apply 30 100 0 4; ./lab.sh flows solo --duration 20
```

Smoothed RTT, each flow alone, buffer 4 × BDP



algorithm (alone)	Mbit/s	retransmits	bottleneck drops	RTT min	RTT mean	RTT max (ms)	MSS	bytes / packet
CUBIC	94.9	63	63	104.5	126.5	150.7	1448	1514
BBR	93.8	0	0	33.1	51.6	73.7	1448	1514
Reno	94.9	1242	1242	44.8	84.6	100.7	1448	1514

The bufferbloat result, in one table. With a 120 ms buffer, CUBIC pushes its mean RTT to 126.46 ms — a 30 ms path made four times slower for everyone sharing the link — with almost no drops (63), because the buffer rarely overflows. Reno's halving drains the queue now and then (mean 84.61 ms). BBR keeps the queue short — mean 51.63 ms with 0 retransmits — at the same throughput (93.79 Mbit/s). This is the case BBR was designed for.

A shallow buffer on a long path: ¼ × BDP at 100 ms, all three at once

```
./lab.sh wan apply 100 100 0 0.25; ./lab.sh flows compete --duration 30
```

algorithm	flow	Mbit/s	retransmits	RTT min	RTT mean	RTT max (ms)	MSS
CUBIC	`a-cubic → b-cubic`	6.8	674	100.9	110.6	126.0	1448
BBR	`a-bbr → b-bbr`	81.7	3297	100.5	109.4	125.2	1448
Reno	`a-reno → b-reno`	3.4	610	100.5	110.1	126.0	1448
total · Jain fairness 0.418		**91.9**	**4581**	**bottleneck drops 4581**			

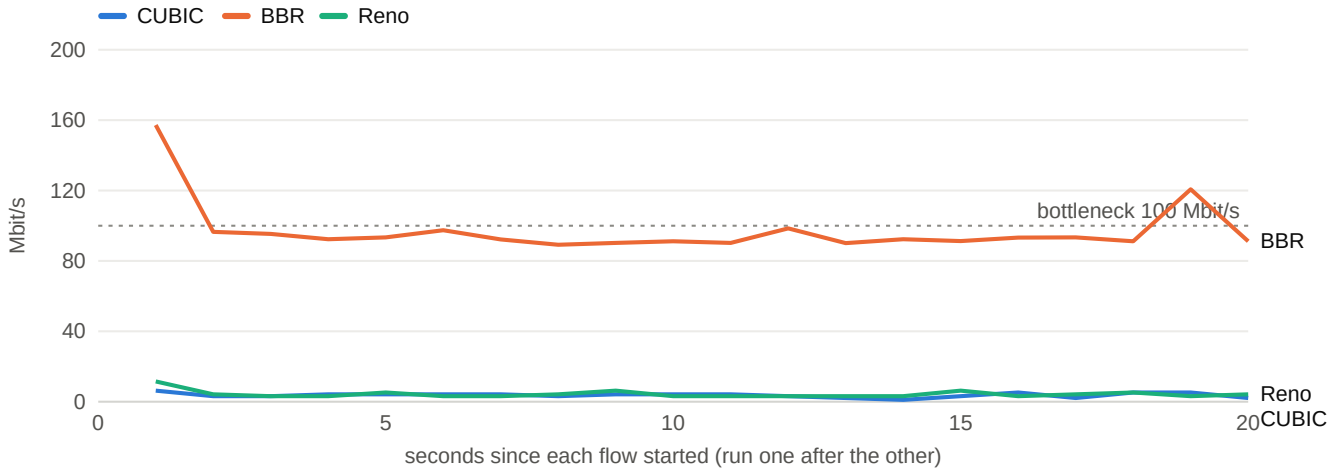
The other end: a quarter-BDP buffer (312 KB, 25 ms) on a 100 ms path. BBR takes 81.71 of 91.92 Mbit/s; CUBIC gets 6.83 and Reno 3.38. A shallow buffer is where loss-based flows suffer most against BBR: every queue that builds overflows quickly, and only one of the three flows backs off when it does.

12. Walkthrough F — random loss

1 % random loss in each direction (netem), otherwise the defaults; each algorithm alone. This loss has nothing to do with congestion — the link stays empty — like a bad radio link.

```
./lab.sh wan apply 30 100 1 1; ./lab.sh flows solo --duration 20
```

Throughput, each flow alone, 1 % random loss each way



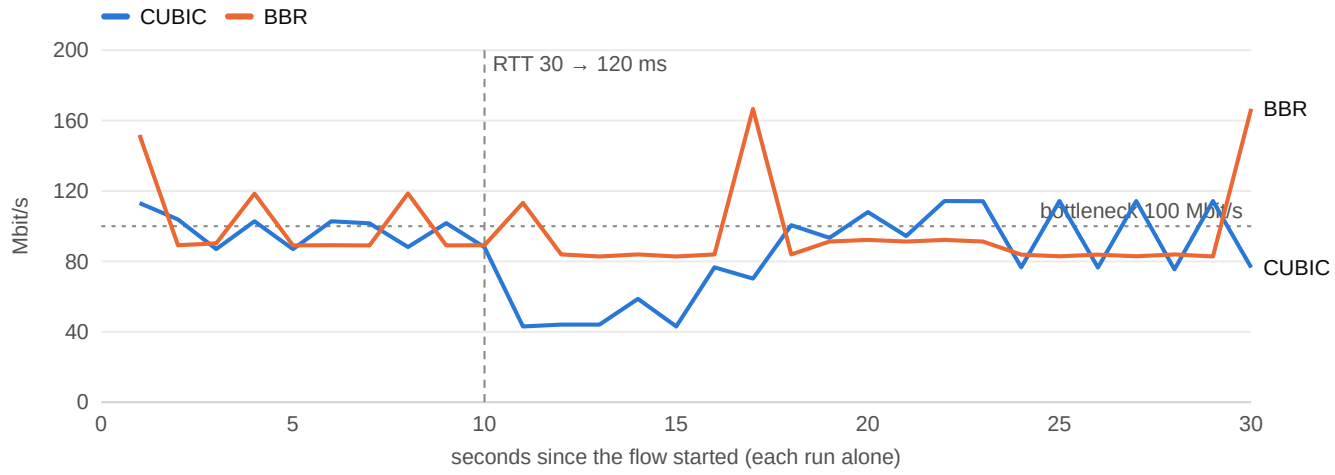
algorithm (alone)	Mbit/s	retransmits	bottleneck drops	RTT min	RTT mean	RTT max (ms)	MSS	bytes / packet
CUBIC	3.7	79	0	30.6	32.2	35.3	1448	1509
BBR	93.3	2956	1321	31.7	42.7	56.5	1448	1514
Reno	4.1	80	0	30.7	32.5	37.0	1448	1510

1 % random loss is devastating for the loss-based algorithms: CUBIC 3.66 and Reno 4.14 Mbit/s on an otherwise empty 100 Mbit/s link — with no bottleneck drops at all and an RTT at the bare path (~32 ms): they never get the window large enough to queue anything. Reno's number is almost exactly the Mathis et al. formula for loss-limited TCP, rate ≈ (MSS / RTT) × 1.22 / √p = (1448 × 8 bit / 32 ms) × 1.22 / 0.1 ≈ 4.4 Mbit/s. CUBIC, built for big windows, does no better here: with a loss every 100 packets its window never leaves the small region where it behaves like Reno. BBR ignores the losses and runs at 93.31 Mbit/s — it even overruns the queue (1321 bottleneck drops on top of the random ones). On lossy radio links this is the whole case for BBR.

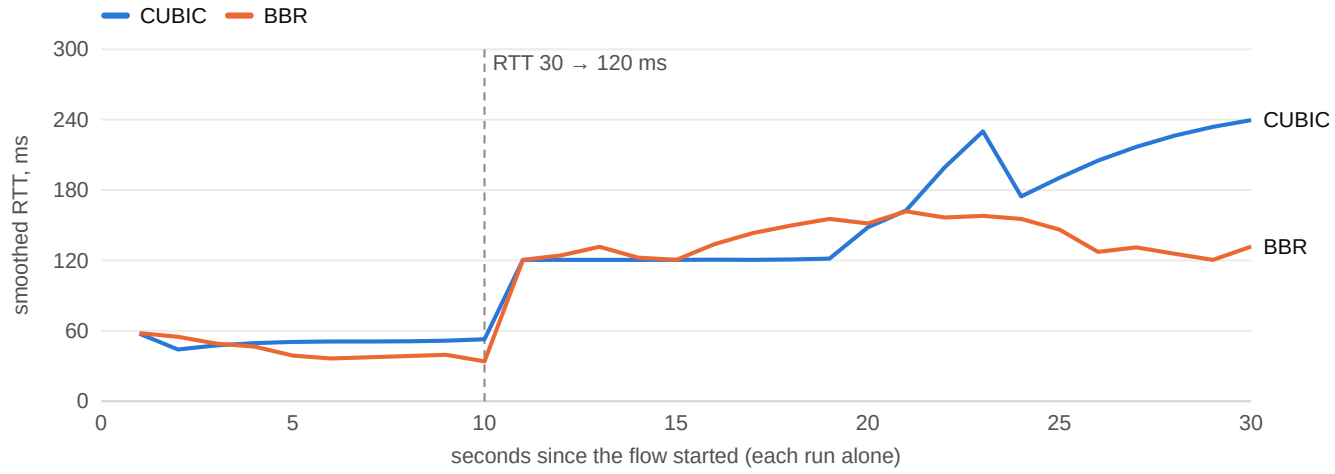
13. Walkthrough G — moving the RTT under a flow

The slider is applied in place, so it can move while a flow is running. One flow alone for 30 s; at 10 s the RTT jumps from 30 to 120 ms (the buffer stays 1 × BDP *of the new RTT*). Once for CUBIC, once for BBR.

Throughput



Smoothed RTT



algorithm	before (s 3–10)	5 s after	last 8 s	whole run	retransmits	error
CUBIC	94.9	46.6	95.3	84.3	210	none
BBR	96.6	89.3	94.8	92.3	899	none

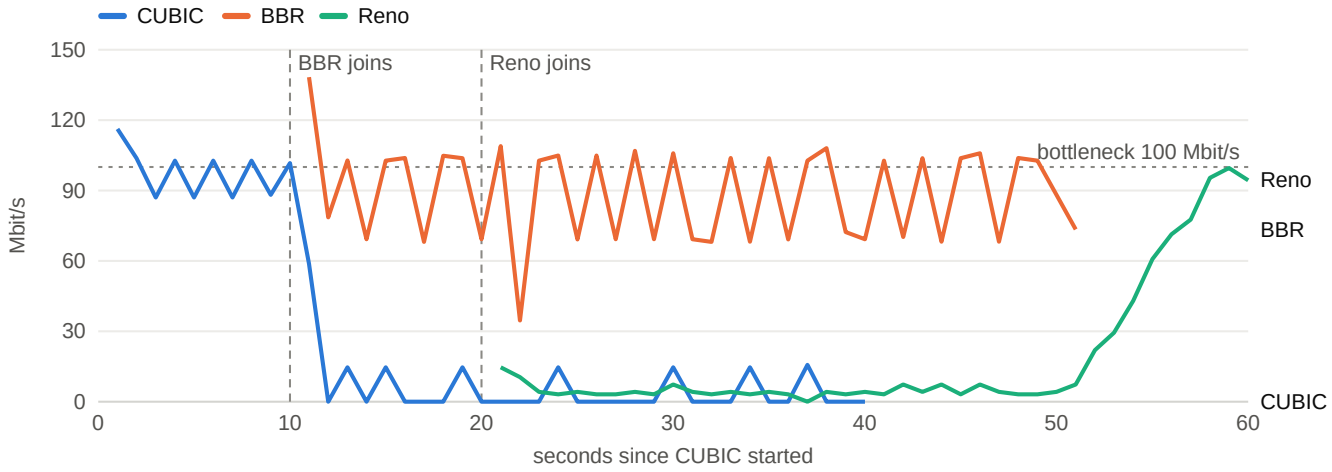
Both flows survive the jump, with no error and no reset. The smoothed RTT steps to ~120 ms within a second of the change — the qdiscs were changed in place, not replaced. CUBIC's throughput halves for about 5 s (its window, sized for 30 ms, now covers only a quarter of the new pipe) and then grows back with its cubic curve; as it does, it fills the new, larger buffer (1 × BDP at 120 ms = 1.5 MB) and the RTT climbs to ~240 ms — the buffer's full drain time on top of the path. BBR's throughput barely moves: it paces at its bandwidth estimate, which the RTT change does not touch, and only its in-flight cap had to grow. Its RTT settles at 120–160 ms.

14. Walkthrough H — late arrivals

CUBIC starts alone; BBR joins 10 s later, Reno 10 s after that (`--stagger 10`). Defaults otherwise. The question: can a newcomer get its share from a flow that already owns the link?

```
./lab.sh flows compete --duration 40 --stagger 10
```

Throughput, staggered starts



algorithm	flow	Mbit/s	retransmits	RTT min	RTT mean	RTT max (ms)	MSS
CUBIC	`a-cubic → b-cubic`	27.7	581	44.5	55.2	61.5	1448
BBR	`a-bbr → b-bbr`	86.9	5387	33.8	53.1	59.7	1448
Reno	`a-reno → b-reno`	18.2	124	30.5	48.8	58.3	1448

The most striking run. CUBIC owns the link for 10 s; when BBR starts, CUBIC is pushed down to nothing within 5 s and reports **0 Mbit/s for most of the next 25 s** — not a small share, a stall: its losses come faster than it can recover and it falls into retransmission timeouts with exponential backoff. Reno, arriving at 20 s, never gets above a few Mbit/s while BBR runs. Only when BBR's 40 s end does Reno ramp up (to ~78 Mbit/s in its last seconds) and CUBIC come back. A newcomer that ignores loss does not just win against an established loss-based flow — it can starve it. (Each flow ran 40 s from its own start, so their averages are not simultaneous and are not summed.)

15. Walkthrough I — jumbo frames

The hosts switch to MTU 9000; nothing else moves. r1, r2 and wanem were 9000 all along.

```
./lab.sh mtu set 9000
```

```
r1       eth1 9000  eth2 9000
r2       eth1 9000  eth2 9000
wanem    eth1 9000  eth2 9000  br0 9000
a-cubic  eth1 9000
a-bbr    eth1 9000
a-reno   eth1 9000
b-cubic  eth1 9000
b-bbr    eth1 9000
b-reno   eth1 9000
```

A full-size, don't-fragment ping (8972 bytes of ICMP payload + 28 bytes of headers = 9000) crosses r1, the emulator and r2:

```
PING 192.168.142.11 (192.168.142.11) 8972(9000) bytes of data.
8980 bytes from 192.168.142.11: icmp_seq=1 ttl=62 time=30.9 ms
8980 bytes from 192.168.142.11: icmp_seq=2 ttl=62 time=30.9 ms
8980 bytes from 192.168.142.11: icmp_seq=3 ttl=62 time=30.9 ms

--- 192.168.142.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 30.908/30.919/30.926/0.008 ms
```

Back at 1500, a 1600-byte don't-fragment ping is refused by the sender's own stack:

```
PING 192.168.142.11 (192.168.142.11) 1600(1628) bytes of data.
ping: sendmsg: Message too large
ping: sendmsg: Message too large

--- 192.168.142.11 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1021ms
```

Each algorithm alone for 15 s at each MTU (defaults otherwise). Bytes per packet are counted at the bottleneck, in wanem:

algorithm (alone, 15 s)	hosts at 1500			hosts at 9000		
	MSS	bytes / packet	Mbit/s	MSS	bytes / packet	Mbit/s
CUBIC	1448	1514	94.7	8948	9005	98.7
BBR	1448	1514	93.1	8948	9005	97.2
Reno	1448	1514	94.5	8948	9004	98.5

At 9000 every algorithm runs faster and loses less: 98–99 Mbit/s against 93–95. The gain is mostly header overhead — 8948 bytes of payload in 9014 on the wire (99.3 %) against 1448 in 1514 (95.6 %) — and partly fewer losses: six times fewer packets means fewer drop events per second, and each window step is six times bigger in bytes. The bytes per packet counted at the bottleneck (9005) prove the jumbo frames really cross r1, wanem and r2 unsplit.

Change the MTU between runs. A TCP connection keeps the MSS it negotiated at its start, so the portal refuses the switch while a run is going. The hosts all switch together: mixed MTUs would only measure path-MTU clamping, and the NMS alerts on it.

16. Walkthrough J — 500 Mbit/s, RTT 0 to 150 ms, each alone, MTU 1500 and 9000

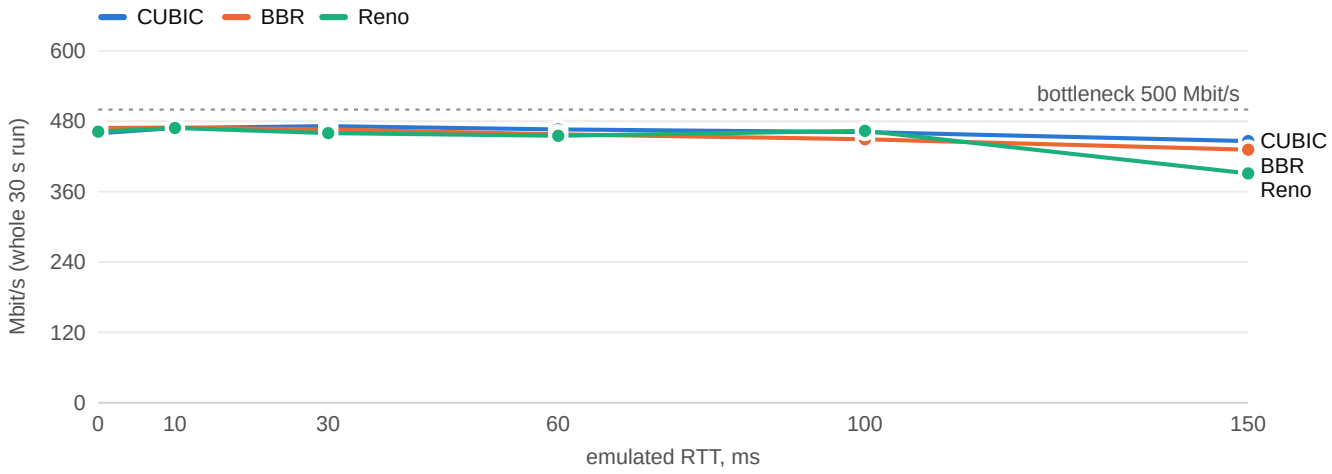
A clean baseline for each algorithm on its own, with nothing to compete against. The bottleneck is raised to **500 Mbit/s**, the RTT is stepped through **0, 10, 30, 60, 100 and 150 ms**, and at every step CUBIC, then BBR, then Reno sends alone. Each run is one TCP flow, `iperf3 -c <peer> -p 5201 -t 30 -b 500M`: a single stream for 30 s, with the application offering 500 Mbit/s. Buffer 1 × BDP (at 0 ms that is zero, so tbf's floor of four jumbo frames, 36 KB, applies), no loss. The series ran twice: first with the hosts at **MTU 1500** (the default), then at **MTU 9000**. The hosts' TCP settings are in section 4 and Appendix D. One command runs a series and puts the WAN back as it found it:

```
./lab.sh mtu set 1500                          # or 9000
./lab.sh flows sweep                           # = --rtts 0,10,30,60,100,150 --rate 500 --duration 30 --bitrate
500M
./lab.sh flows sweep --algos reno --rtts 100   # any subset
```

The Robot suite `05_rtt_series` runs the same series (same iperf3 options) as one test per algorithm and checks the rate at every RTT (section 20).

Hosts at MTU 1500

Throughput of each algorithm alone, by RTT (bottleneck 500 Mbit/s, iperf3 -t 30 -b 500M, MTU 1500)

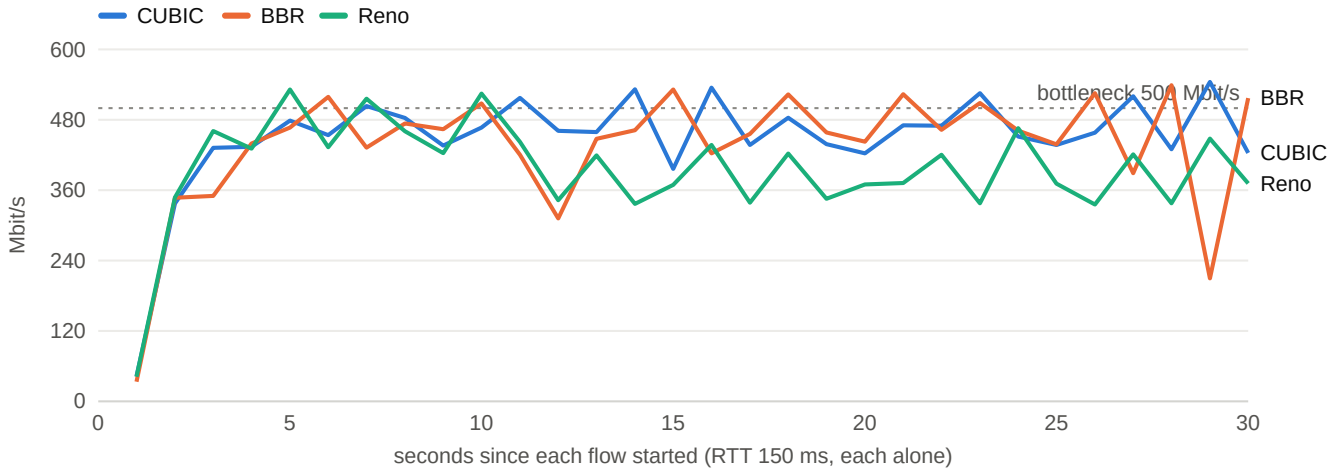


RTT set (ms)	ping avg (ms)	buffer (MB)	CUBIC			BBR			Reno		
			Mbit/s	retr	RTT mean	Mbit/s	retr	RTT mean	Mbit/s	retr	RTT mean
0	0.8	0.04	459.5	6729	0.9	468.4	91894	1.1	462.2	5079	1.0
10	10.9	0.62	468.4	1050	15.4	469.2	50491	19.8	468.6	1372	13.8
30	31.0	1.88	471.7	1974	51.4	465.9	33073	55.8	459.8	2529	37.2
60	61.0	3.75	466.1	5919	94.7	457.8	16607	97.1	455.3	3723	66.3
100	101.0	6.25	461.5	1728	159.2	449.4	2641	144.6	463.5	3167	165.5
150	151.0	9.38	446.1	2343	185.8	431.5	3120	163.6	390.9	3069	165.8

ping avg: a-cubic → its peer, measured before the flows at each step (the set RTT plus the VMs' own ~1 ms). Buffer: tbf's queue, 1 × BDP at 500 Mbit/s. retr: TCP retransmits over the 30 s.

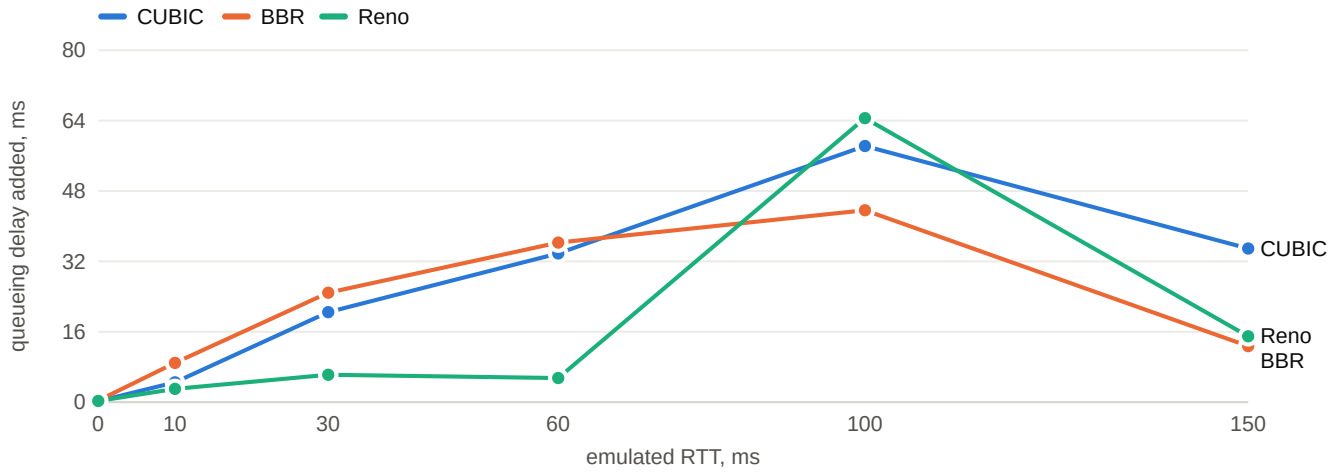
Up to 100 ms the algorithm does not matter for throughput. All three carry 449–472 Mbit/s. That is the ceiling: tbf counts every 1514-byte frame against the 500 Mbit/s, and iperf3 reports the 1448 bytes of payload in it, so ~478 Mbit/s of goodput is the most any TCP can show here. What differs is the cost. BBR gets the same rate with far more retransmits (91894 at 0 ms against 6729 for CUBIC, and 50491 against 1050 at 10 ms), because it probes for bandwidth by pacing above its estimate and tolerates the losses.

Per second at RTT 150 ms, each alone



At 150 ms Reno falls behind. All three ramp up in a few seconds (tens of RTTs to reach the 9.4 MB BDP). CUBIC and BBR then hold 430–540 Mbit/s. Reno does too until around second 12, when a loss halves its window; from there it saw-tooths between ~340 and ~440 Mbit/s, averaging 390.89 against CUBIC's 446.09. Reno adds one segment per RTT, which at 150 ms is under 7 segments per second; refilling half of a 9.4 MB BDP takes ~3 200 segments, about eight minutes. CUBIC's window grows with the time since the loss, not with the number of RTTs, which is the problem it was built to fix. How much this costs Reno depends on when its losses fall: in an earlier 20 s run of this series, Reno met one loss at 100 ms and stayed near 260 Mbit/s for the rest of the run.

Queueing delay each flow added on top of the idle path (mean smoothed RTT − ping min)

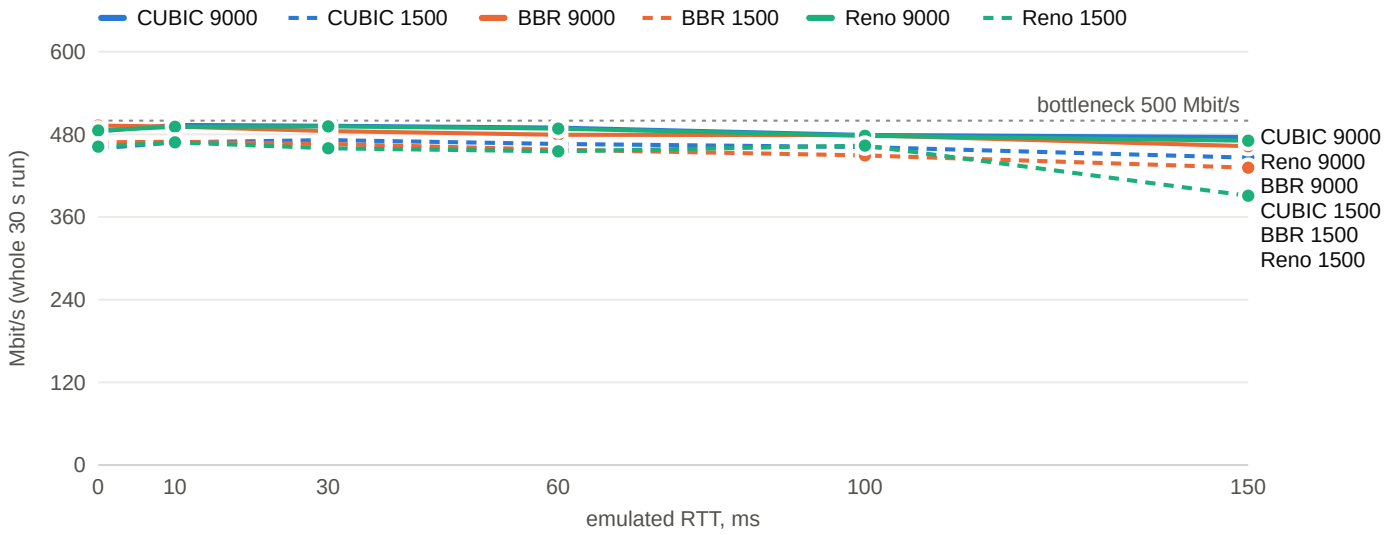


The queue tells the algorithms apart more than the throughput does. Up to 60 ms Reno keeps the smallest standing queue (3–6 ms), while at 30 and 60 ms CUBIC and BBR add 20–36 ms. BBR's probing pushes 25 % above its estimate, which shows with a 1 × BDP buffer. On the long paths BBR adds the least: 163.61 ms mean RTT at 150 ms against CUBIC's 185.78. It aims to keep the bottleneck queue near empty, and on a long path a full buffer is a lot of extra delay.

Hosts at MTU 9000

The same series, with the same iperf3 options, with the hosts switched to jumbo frames (MSS 8948); r1, r2 and wanem were 9000 throughout. Solid lines: 9000; dashed: the 1500 run above.

Throughput by RTT, MTU 9000 (solid) against 1500 (dashed)



RTT (ms)	CUBIC				BBR				Reno			
	1500	9000	gain	retr 1500/9000	1500	9000	gain	retr 1500/9000	1500	9000	gain	retr 1500/9000
0	459	484	+5 %	6729 / 7568	468	493	+5 %	91894 / 18628	462	486	+5 %	5079 / 10841
10	468	494	+5 %	1050 / 214	469	491	+5 %	50491 / 5620	469	491	+5 %	1372 / 215
30	472	492	+4 %	1974 / 330	466	485	+4 %	33073 / 2322	460	491	+7 %	2529 / 451
60	466	490	+5 %	5919 / 673	458	480	+5 %	16607 / 1166	455	488	+7 %	3723 / 730
100	462	479	+4 %	1728 / 221	449	479	+7 %	2641 / 0	464	478	+3 %	3167 / 585
150	446	476	+7 %	2343 / 0	432	463	+7 %	3120 / 0	391	471	+20 %	3069 / 78

Mbit/s over 20 s, each alone, bottleneck 500 Mbit/s, buffer 1 × BDP; hosts at MTU 1500 (MSS 1448) and 9000 (MSS 8948). Two separate series, run one after the other.

At 9000 every algorithm sits at the ceiling at every RTT: 463–494 Mbit/s from 0 to 150 ms, against 391–472 at 1500. Most of the gain is the higher ceiling. 8948 bytes of payload per 9014 on the wire is 99.3 %, so ~496 Mbit/s of goodput against ~478 at 1500; that accounts for 4–5 % at every RTT. The rest comes from losing less:

- **Retransmits fall 5–14× from 10 to 60 ms**, and at 100–150 ms CUBIC and BBR retransmit nothing. At 0 ms only BBR's fall: the 36 KB buffer holds only four jumbo frames, and CUBIC and Reno retransmit as much or more. BBR goes from 33073 to 2322 at 30 ms. Six times fewer packets for the same bytes means six times fewer packets to queue, burst and drop.
- **Reno keeps up at 150 ms** (470.88 Mbit/s against 390.89 at 1500, +21 %). Each one-segment-per-RTT step now adds six times as many bytes: recovering half the BDP takes ~520 RTTs instead of ~3 200.
- **Slow start reaches the BDP sooner.** Each doubling is in bigger segments, and the initial window of 10 segments is 89 KB instead of 14 KB.

> **Why 500 Mbit/s and not 100.** At 500 Mbit/s the BDP at 150 ms grows to 9.4 MB. That is where window-growth rules (Reno's linear increase, the length of slow start) start to show in a single flow. The hosts' 32 MB socket buffers cover it on paper, but their small tcp_mem does not always (section 4).

17. Walkthrough K — is BBR better on long fat networks?

The claim to test: *on paths with high bandwidth and high latency, BBR delivers more throughput than the loss-based algorithms.* This section tests it as a controlled comparison: one variable (the algorithm), everything else held identical and verified, and the path conditions under which the claim should and should not hold, chosen before the run.

Design

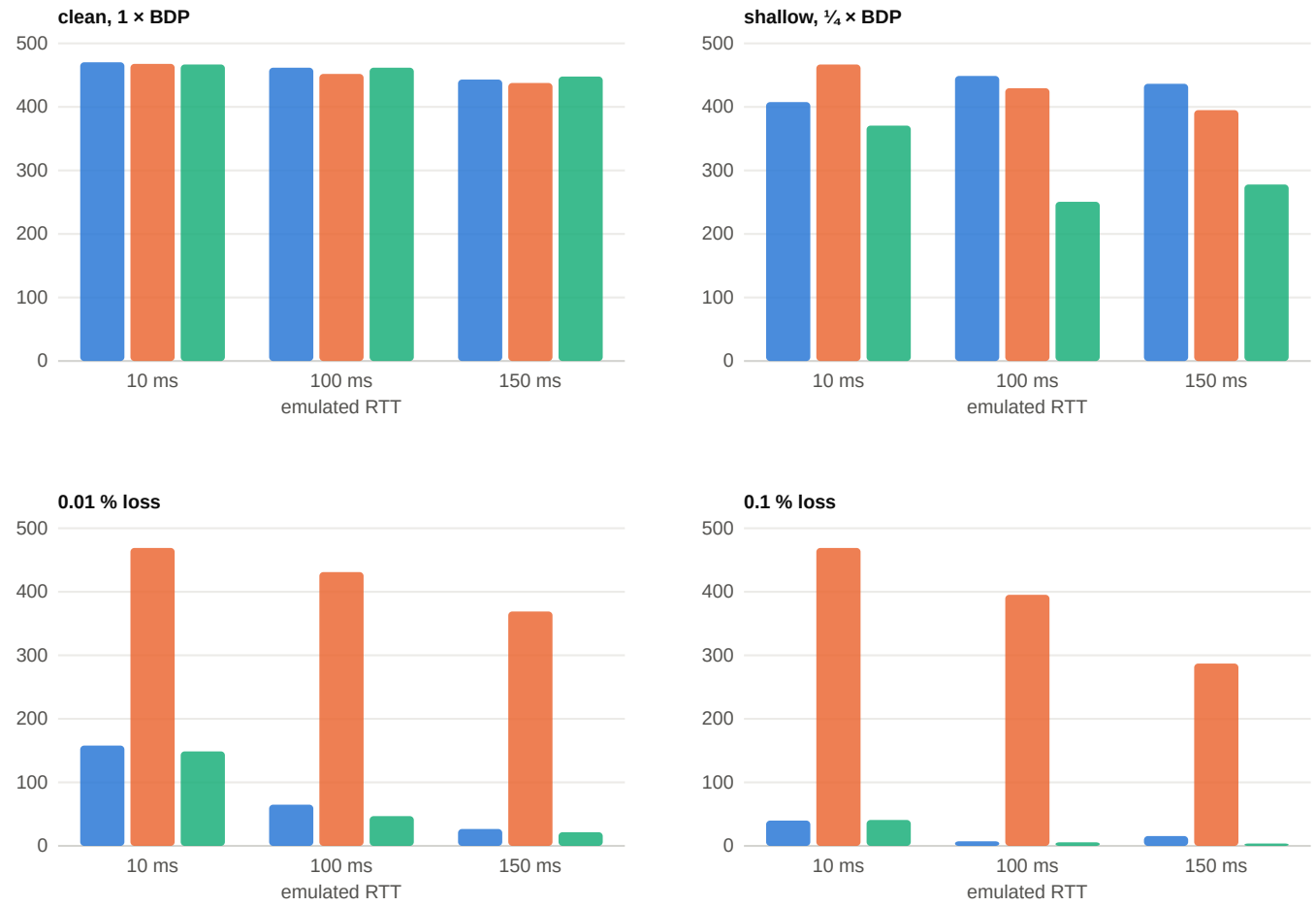
element	choice
Question	Does the congestion control algorithm change the throughput of a single bulk TCP flow on a high-bandwidth, high-latency path, and under which path conditions?
Treatment (what varies)	The sender's congestion control: CUBIC, BBR (v1) or Reno. Each is a separate host (a-cubic, a-bbr, a-reno) sending to its own peer across the same routers, emulator and bottleneck.
Path conditions (factors)	Bottleneck **500 Mbit/s**; RTT **10 ms** (a short-path reference), **100 ms** and **150 ms**; four path types: **clean** (1 × BDP buffer, no loss), **shallow buffer** (¼ × BDP), **0.01 % random loss** and **0.1 % random loss** (1 × BDP). 3 × 4 = 12 cells.
Controlled (identical for all three)	Everything on the hosts: the same image, kernel, every TCP and socket sysctl (section 4), MTU 1500, fq on eth1, iperf3 version and options. Nothing on the hosts was changed for the experiment. Only one flow at a time crosses the bottleneck.
Traffic	One TCP flow at a time across the bottleneck: `iperf3 -c <peer> -p 5201 -t 30 -b 500M`, a single stream (no `-P`) for 30 s with the application offering 500Mbit/s, from a-<algorithm> to b-<algorithm>.
Replication	One run per algorithm per profile: 36 runs in all, about 21 minutes.
Order	Within each profile the three algorithms run back to back in a random order (seeded, seed 1), 2 s apart.
Measurement	Receiver-side goodput reported by iperf3 over the whole run (start-up included); also retransmits, smoothed RTT, bottleneck drops and the sender's TCP memory-pressure events.
Analysis	Per profile: each algorithm's goodput, and BBR's as a ratio of CUBIC's and Reno's. With one run per profile there is no within-profile variance to test against, so differences under ~5 % (the run-to-run spread seen in sections 16 and 20) count as no difference.
Integrity checks	Before and after the experiment every host's TCP and socket sysctls, eth1 MTU and qdisc were fingerprinted (SHA-256 over 121 settings). Before: all six hosts identical apart from the algorithm: **True** (fingerprint 7fd0557c5b315478). After: unchanged: **True**. Every run's reported algorithm matched its host; failed runs: 0.

Hypotheses, written down before the run:

- **H1 — clean path:** no meaningful difference. With no loss and a 1 × BDP buffer every algorithm can fill the pipe (section 16), so BBR should not win.
- **H2 — random loss:** BBR is much faster, and the gap grows with RTT. A loss-based sender halves (Reno) or cuts by 30 % (CUBIC) its window on every loss, and on a long path it needs many round trips to grow back; BBR does not treat an isolated loss as congestion.
- **H3 — shallow buffer:** BBR is faster on long paths. With a buffer smaller than the BDP, a loss-based sender's sawtooth falls below the pipe size after every overflow; BBR paces at the measured bottleneck rate and does not need the buffer.

```
python3 tools/bbr_experiment.py --trials 1 --duration 30 --bitrate 500M     # ~21 min; resumable; restores the
WAN
```

Results



path	RTT (ms)	CUBIC	BBR	Reno (Mbit/s)	BBR ÷ CUBIC	BBR ÷ Reno
clean, 1 × BDP	10	470.2	467.8	466.7	0.99×	1.00×
shallow, ¼ × BDP	10	407.7	466.7	370.4	**1.14×**	**1.26×**
0.01 % loss	10	157.9	469.1	148.6	**2.97×**	**3.16×**
0.1 % loss	10	39.8	469.1	40.9	**11.80×**	**11.48×**
clean, 1 × BDP	100	461.8	451.9	461.6	0.98×	0.98×
shallow, ¼ × BDP	100	448.6	429.4	250.5	0.96×	**1.71×**
0.01 % loss	100	65.0	430.9	46.9	**6.63×**	**9.19×**
0.1 % loss	100	7.2	395.1	5.8	**54.50×**	**67.54×**
clean, 1 × BDP	150	443.0	437.8	447.7	0.99×	0.98×
shallow, ¼ × BDP	150	436.5	394.8	277.7	0.90×	**1.42×**
0.01 % loss	150	26.5	369.0	21.4	**13.93×**	**17.23×**
0.1 % loss	150	15.4	287.1	3.8	**18.68×**	**76.37×**

Mbit/s: iperf3 receiver goodput over the 30 s run. Ratio: BBR over the other algorithm; bold where BBR is more than 5 % ahead.

H2 holds, strongly: with random loss BBR wins by a wide margin, and at 0.01 % loss the margin grows with the RTT. At 0.01 % loss (one packet in ten thousand) BBR carries 469, 431 and 369 Mbit/s at 10, 100 and 150 ms; CUBIC

carries 158, 65 and 26. That is 3.0×, 6.6× and 13.9× CUBIC's throughput. At 0.1 % loss CUBIC and Reno collapse to 7 and 6 Mbit/s at 100 ms, while BBR keeps 395. (At 0.1 % CUBIC and Reno are already down to a few Mbit/s at 100 ms, so their 150 ms values, single-digit or low-teens Mbit/s, do not show a clean trend.) The mechanism is the one the hypothesis named. Every loss cuts a loss-based window (CUBIC by 30 %, Reno by half), and on a long path the window takes many round trips to grow back, so the next loss arrives first. BBR sets its rate from the delivery rate and the minimum RTT it measures, and does not treat an isolated loss as congestion. Its throughput falls with the RTT too (469 → 287 Mbit/s at 0.1 %), but far less.

H1 holds: on a clean path BBR is not better. With no loss and a 1 × BDP buffer, all three fill the pipe at every RTT: 462 / 452 / 462 Mbit/s (CUBIC / BBR / Reno) at 100 ms and 443 / 438 / 448 at 150 ms. The differences are 1–2 %, within run-to-run noise. BBR's advantage on long fat networks is not raw speed on a perfect path.

H3 holds only against Reno. With a buffer of ¼ × BDP, BBR beats Reno at every RTT (1.7× at 100 ms, 1.4× at 150 ms) and beats CUBIC at 10 ms. At 100 and 150 ms, though, CUBIC was ahead (449 against 429, and 437 against 395). A plausible reading: CUBIC backs off only 30 % per loss and regrows on a timer rather than per RTT, so a shallow buffer costs it little, while BBR's probing overflows the small buffer (its retransmits are 2–8× CUBIC's here). The hypothesis is therefore only partly supported.

> **Conclusion.** On high-bandwidth, high-latency paths BBR is decisively better *when the path loses packets for reasons other than congestion*: 13.9× CUBIC at 0.01 % loss and 150 ms, and 54.5× at 0.1 % and 100 ms. That covers wireless and satellite links, long-haul circuits with a low bit-error rate, and policers. With 0.01 % loss its advantage grows with the RTT, the case where loss-based algorithms are weakest. On a clean path with an adequately sized buffer it is no faster than CUBIC, and in a shallow buffer CUBIC matched or beat it on long paths.

What BBR pays for it

path	RTT	retransmits per run (mean)			queueing delay added, ms (mean)			TCP memory pressure events (sum)		
		CUBIC	BBR	Reno	CUBIC	BBR	Reno	CUBIC	BBR	Reno
clean, 1 × BDP	10	1200	55909	776	5.1	7.3	3.4	0	12	1
shallow, ¼ × BDP	10	1201	83698	1068	0.2	1.9	0.1	0	4	0
0.01 % loss	10	59	49238	1012	0.0	8.0	0.0	0	7	0
0.1 % loss	10	118	43224	123	0.0	8.4	0.0	0	1	0
clean, 1 × BDP	100	1519	2545	2407	59.8	42.1	52.5	151	152	155
shallow, ¼ × BDP	100	3544	29407	3094	5.7	10.1	0.0	154	141	3
0.01 % loss	100	379	3264	2503	0.0	12.4	1.0	13	134	5
0.1 % loss	100	19	2914	13	1.2	6.5	0.2	0	123	0
clean, 1 × BDP	150	3702	941	3230	37.0	15.9	38.9	148	143	150
shallow, ¼ × BDP	150	6768	11968	5801	10.6	7.1	0.0	147	119	96
0.01 % loss	150	9	2101	8	0.0	7.0	0.0	0	106	2
0.1 % loss	150	27	1865	12	0.7	5.1	0.1	1	90	0

BBR's throughput is not free. At short RTTs it retransmits far more: 55,909 retransmits on the clean 10 ms path against CUBIC's 1,200. BBR v1 probes at 125 % of its estimate and does not slow down on loss, so a 1 × BDP buffer overflows on every probe. On long paths the picture reverses for delay: BBR adds less standing queue than CUBIC and Reno on the clean path, because it aims to keep the bottleneck queue near empty. All three algorithms hit the hosts' TCP memory limit on the long clean paths (about 150 pressure events per run each), so that limit capped them alike. In the loss cells only BBR got fast enough to reach it.

Threats to validity

- **Emulated loss is random and independent.** Real loss is often bursty or caused by congestion elsewhere. Random loss is exactly the case where BBR's design helps most, so H2 is the most favourable test for BBR; the clean and shallow-buffer cells are the fairer comparison.
- **One flow at a time.** This is a throughput test for a single transfer. When BBR shares a link with CUBIC or Reno it takes far more than its fair share (sections 9 and 14). Being "better" for your own transfer and being a good neighbour are different questions.
- **BBR v1.** Linux 6.12 ships BBR v1. BBRv3 reacts to loss above about 2 %, retransmits less, and would give smaller gains in the loss cells.
- **Small host VMs.** The hosts' TCP memory limit (section 4) is the same for every algorithm, and pressure events are reported in the cost table. It caps what any algorithm can reach on the longest paths, so it narrows the differences rather than creating them.
- **One run per profile.** Large effects (several times the throughput) are far outside run-to-run noise; small ones (a few %) are not, and the text does not claim them.
- **30 s runs include start-up.** At 150 ms slow start takes a few seconds of each run, the same for every algorithm. Longer transfers would bring the averages closer to the steady-state rates.

18. The portal

`http://192.168.50.231:8112` (user unit `tcp-cc-lab-webapp`, FastAPI on the shared `labportal` package, Lab Hub sign-in). Below, a compete run started from the portal, with the RTT moved from 30 to 100 ms at 16 s and back at 31 s with the slider — the dashed rules on the charts are those changes.

- **WAN emulator card:** the RTT slider (0–300 ms; applied when released, in place), rate, loss, buffer, the host MTU switch, and a ping button. Live tx rate, queue backlog and drops for both directions, polled every 2 s.
- **Flows:** compete or solo, duration, direction, which algorithms, stagger. Two live charts — throughput and smoothed RTT per flow — drawn from the run's `SAMPLE` lines, with a crosshair tooltip; RTT changes during a run are marked. The result table follows (Mbit/s, retransmits, RTT, MSS, Jain).
- **Topology, hosts table** (algorithm, MTU, fq, iperf3 per host), **tests** (run all suites or one; the last report), **recent runs**, and a light/dark switch.

tcp-cc-lab TCP congestion control (CUBIC, BBR, Reno) across an emulated WAN v0.3.0

lab running ☾ dark API Lab Hub

WAN emulator — wanem: netem delay (RTT ÷ 2 each way) then a tbf bottleneck, both directions

Round-trip time **30 ms** 15 + 15

Bottleneck rate 100 Mbit/s

Random loss **0 %** each way

Bottleneck buffer ¼ BDP | 1 BDP | 4 BDP | 16 BDP = 375 kB (30.0 ms at 100 Mbit/s)

Host MTU 1500 | 9000 (jumbo) r1, r2, wanem links: 9000 ✓

Defaults (30 ms, 100 Mbit/s) Measure RTT

A → B through the bottleneck	Queue at the bottleneck (A → B)	Bottleneck drops (A → B)	B → A (acks, or B senders)
–	0 kB · 0.0 ms	–	–

The MTU switch sets all six hosts' eth1 (between runs: a flow keeps the MSS it started with); r1, r2 and wanem's links are always 9000. Changes apply when you let go of the slider, in place (`tc qdisc` change): a running flow sees the new RTT on its next packets and keeps going. The VMs add about 1 ms of their own on top of the emulated RTT.

Topology



site A 192.168.141/24 · WAN 100.64.14.0/30 · site B 192.168.142/24

Hosts

Node	Role	Data IP	CC	MTU	State
r1	router	192.168.141.1	–	9000	running
r2	router	192.168.142.1	–	9000	running
wanem	wanem	bridge	–	9000	running
a-cubic	host	192.168.141.11	CUBIC	1500	running
a-bbr	host	192.168.141.12	BBR	1500	running
a-reno	host	192.168.141.13	Reno	1500	running
b-cubic	host	192.168.142.11	CUBIC	1500	running
b-bbr	host	192.168.142.12	BBR	1500	running
b-reno	host	192.168.142.13	Reno	1500	running

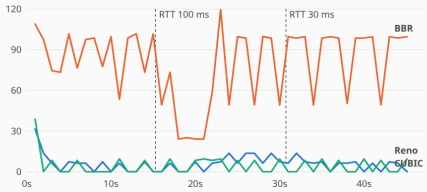
Flows — one TCP flow per algorithm, each site A host to its site B peer (iperf3)

Compete (all at once) | Solo (one at a time) duration 30 s direction site A → B start together

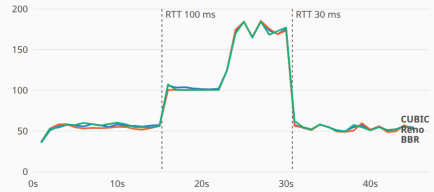
CUBIC BBR Reno Start

CUBIC (a-cubic) BBR (a-bbr) Reno (a-reno)

Throughput Mbit/s, per second



RTT the sender sees ms (smoothed RTT, TCP_INFO)



All at once, 45 s each, site A → B; WAN at the start: RTT 30 ms, 100 Mbit/s, loss 0 %, buffer 1 × BDP; host MTU 1500 (MSS 1448)

Algorithm	Sender	Mbit/s	Share	Retransmits	RTT min	mean	max (ms)
CUBIC	a-cubic	5.77	7 %	384	36.21	82.33	184.77
BBR	a-bbr	77.21	89 %	4271	37.51	81.26	185.29
Reno	a-reno	3.91	4 %	354	37.01	82.25	184.49

Total **86.89** Mbit/s · Jain fairness **0.419** (1 = equal shares, 0.33 = one flow takes all) · bottleneck drops **5034**

Verify and tests

Verify Run tests

19/19 passed results_2026-10-22_23-10-09 · report · log

01 Platform	5/5
02 Wan	5/5
03 Cc	4/4
04 Mtu	5/5

Run

compete success 2026-10-23_00-43-18-2889

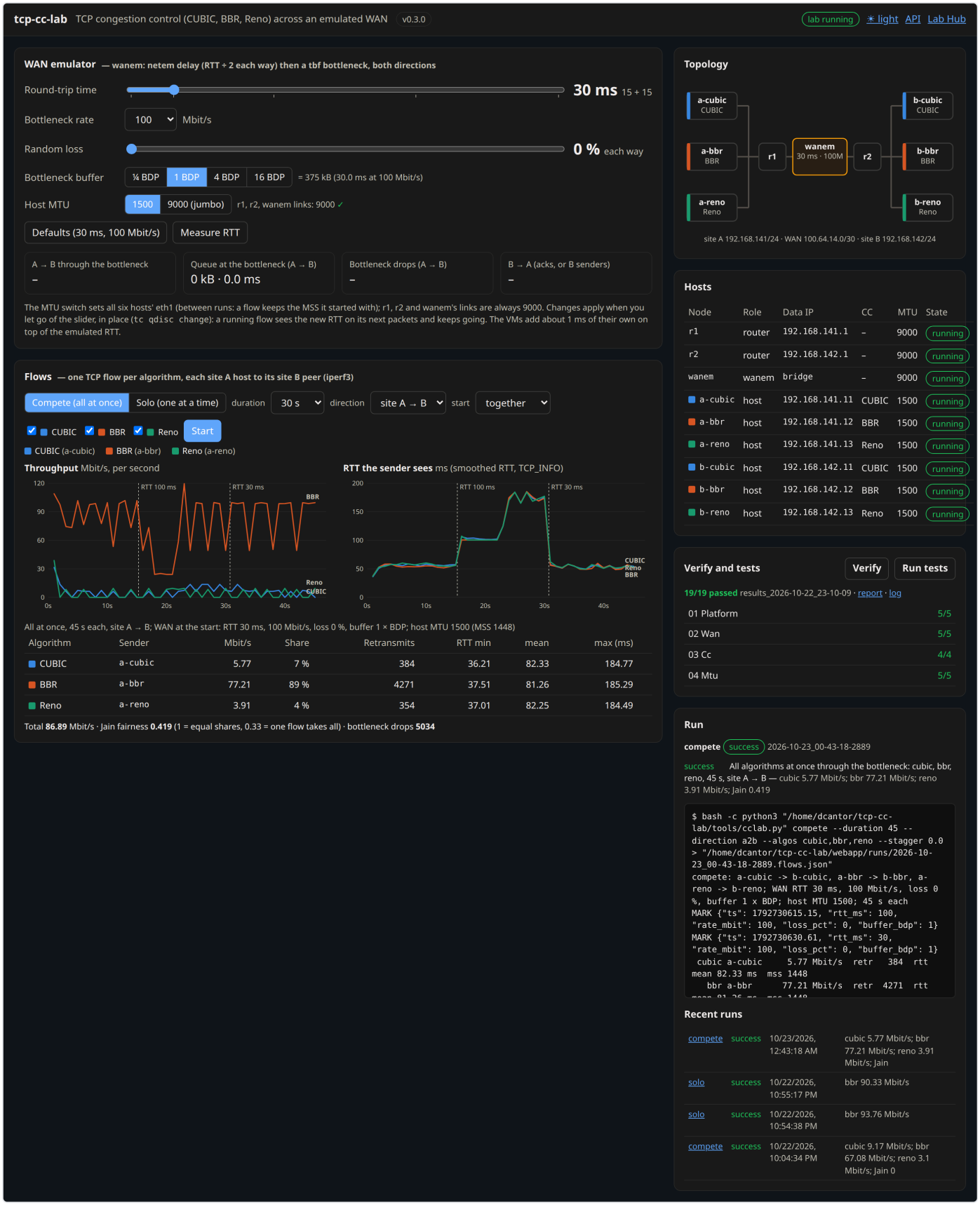
success All algorithms at once through the bottleneck: cubic, bbr, reno, 45 s, site A → B — cubic 5.77 Mbit/s; bbr 77.21 Mbit/s; reno 3.91 Mbit/s; Jain 0.419

```
$ bash -c python3 "/home/dcantor/tcp-cc-lab/tools/cclab.py" compete --duration 45 --direction a2b --algos cubic,bbr,reno --stagger 0.0 > "/home/dcantor/tcp-cc-lab/webapp/runs/2026-10-23_00-43-18-2889.flows.json"
compete: a-cubic -> b-cubic, a-bbr -> b-bbr, a-reno -> b-reno; WAN RTT 30 ms, 100 Mbit/s, loss 0 %, buffer 1 x BDP; host MTU 1500; 45 s each
MARK {"ts": 1792730615.15, "rtt_ms": 100, "rate_mbit": 100, "loss_pct": 0, "buffer_bdp": 1}
MARK {"ts": 1792730630.61, "rtt_ms": 30, "rate_mbit": 100, "loss_pct": 0, "buffer_bdp": 1}
 cubic a-cubic     5.77 Mbit/s  retr   384  rtt mean 82.33 ms  mss 1448
   bbr a-bbr      77.21 Mbit/s  retr  4271  rtt
```

Recent runs

compete	success	10/23/2026, 12:43:18 AM	cubic 5.77 Mbit/s; bbr 77.21 Mbit/s; reno 3.91 Mbit/s; Jain
solo	success	10/22/2026, 10:55:17 PM	bbr 90.33 Mbit/s
solo	success	10/22/2026, 10:54:38 PM	bbr 93.76 Mbit/s
compete	success	10/22/2026, 10:04:34 PM	cubic 9.17 Mbit/s; bbr 67.08 Mbit/s; reno 3.1 Mbit/s; Jain 0

Light theme. The live tiles under the WAN controls (rate, queue, drops) are empty because this is a copy of the page fed with the run's recorded API answers — on the live page they update every 2 s.



Dark theme (the switch top right; remembered per browser).

API

endpoint	what
`GET /api/lab, /api/state`	the inventory and limits; nodes' health, WAN, MTU, last tests, last result, the run going
`GET / POST /api/wan`	settings and both directions' counters / change any of rtt_ms, rate_mbit, loss_pct, buffer_bdp (marks a running flows run)
`GET / POST /api/mtu`	the hosts' and infrastructure MTU / `{"mtu": 1500|9000}` (refused while a run goes)
`POST /api/rtt`	ping a-cubic → b-cubic now
`POST /api/runs, GET /api/runs[/id]`	start compete | solo | verify | test; status, steps and log (incremental with `?since=`)
`GET /metrics, /api/sd`	Prometheus metrics and HTTP service discovery
`/report/{name}`	the last Robot report.html / log.html

19. Monitoring and the Lab Hub

NMS

The NMS's Prometheus discovers the portal through `http://10.0.0.1:8112/api/sd` (job `tcp-cc-lab`, every 15 s) and scrapes `/metrics`. Its config lives in `~/lab-portal/monitoring` (deployed with `deploy.sh`). Service discovery answer:


```
[
 {
  "targets": [
   "10.0.0.1:8112"
  ],
  "labels": {
   "lab": "tcp-cc-lab",
   "job": "portal",
   "role": "portal"
  }
 }
]
```


A selection of the metrics:


```
tcpcc_wan_rtt_ms{lab="tcp-cc-lab"} 30
tcpcc_wan_rate_mbit{lab="tcp-cc-lab"} 100
tcpcc_wan_loss_pct{lab="tcp-cc-lab"} 0
tcpcc_wan_buffer_bdp{lab="tcp-cc-lab"} 1
tcpcc_wan_queue_bytes{lab="tcp-cc-lab"} 375000
tcpcc_bottleneck_bytes_total{lab="tcp-cc-lab",direction="a2b"} 10433479997
tcpcc_bottleneck_bytes_total{lab="tcp-cc-lab",direction="b2a"} 449571010
tcpcc_bottleneck_drops_total{lab="tcp-cc-lab",direction="a2b"} 56430
tcpcc_bottleneck_drops_total{lab="tcp-cc-lab",direction="b2a"} 2414
tcpcc_bottleneck_backlog_bytes{lab="tcp-cc-lab",direction="a2b"} 0
tcpcc_bottleneck_backlog_bytes{lab="tcp-cc-lab",direction="b2a"} 0
tcpcc_host_mtu{lab="tcp-cc-lab",node="a-cubic"} 1500
tcpcc_host_mtu{lab="tcp-cc-lab",node="a-bbr"} 1500
tcpcc_host_mtu{lab="tcp-cc-lab",node="a-reno"} 1500
tcpcc_host_mtu{lab="tcp-cc-lab",node="b-cubic"} 1500
tcpcc_host_mtu{lab="tcp-cc-lab",node="b-bbr"} 1500
tcpcc_host_mtu{lab="tcp-cc-lab",node="b-reno"} 1500
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="a-cubic",expected="cubic",cc="cubic"} 1
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="a-bbr",expected="bbr",cc="bbr"} 1
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="a-reno",expected="reno",cc="reno"} 1
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="b-cubic",expected="cubic",cc="cubic"} 1
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="b-bbr",expected="bbr",cc="bbr"} 1
tcpcc_host_cc_ok{lab="tcp-cc-lab",node="b-reno",expected="reno",cc="reno"} 1
tcpcc_last_compete_mbps{lab="tcp-cc-lab",cc="cubic"} 9.17
tcpcc_last_compete_share{lab="tcp-cc-lab",cc="cubic"} 0.116
tcpcc_last_compete_mbps{lab="tcp-cc-lab",cc="bbr"} 67.08
tcpcc_last_compete_share{lab="tcp-cc-lab",cc="bbr"} 0.845
tcpcc_last_compete_mbps{lab="tcp-cc-lab",cc="reno"} 3.1
tcpcc_last_compete_share{lab="tcp-cc-lab",cc="reno"} 0.039
tcpcc_last_compete_jain{lab="tcp-cc-lab"} 0.457
tcpcc_lab_run_active{lab="tcp-cc-lab"} 0
```

alert (group tcp-cc-lab)	fires when
TcpCcWanEmulatorUnreachable	the portal can't read wanem's counters
TcpCcHostDown	a host doesn't answer SSH
TcpCcWrongAlgorithm	a host's kernel isn't using the algorithm its name says
TcpCcHostNotReady	no fq on eth1, or iperf3 not listening
TcpCcInfraMtuNot9000	an r1 / r2 / wanem data link isn't 9000
TcpCcHostMtuMixed	the hosts don't all have the same MTU
TcpCcTestsFailing	the last Robot run had failures

Grafana dashboard **tcp-cc-lab: overview** (generated by `gen_dashboards.py`): the WAN settings, both directions' bottleneck rate, backlog and drops, per-host health and MTU, and the last compete result per algorithm.

Lab Hub and fast resume

The hub's card starts and stops the lab. With `"fast_resume": true` its shutdown saves each VM's memory (`virsh managedsave`, ~3.1 GB, ~10 s) and the next start resumes them and runs `./lab.sh resumed`: wait for SSH, set the clocks (they stood still while saved), verify — about 35 s in all, against several minutes from cold. The WAN emulator's qdiscs, the hosts' sysctls and MTU, and the routes come back with the memory. `./lab.sh up` boots cold if a saved image will not restore; `rebuild` and `clean` discard it.

20. Tests

Robot Framework, five suites (`./lab.sh test`, or the portal's Tests card). Keywords are Python (`tests/resources/TcpCc.py`, on top of `tools/cclab.py`); every suite starts from the defaults and the hosts at 1500, and restores them.

suite	passed	time	tests
01 Platform	5/5	8 s	Every node answers on its management address, Each host runs the algorithm its name says, with BBR available everywhere, Hosts pace with fq, keep no cached metrics, and have buffers for 300 ms at 100 Mbit/s, The routers route each site's LAN across the WAN, wanem bridges r1 and r2 with offloads off, so its queues count real packets
02 Wan	5/5	79 s	The emulator starts at the lab defaults: 30 ms RTT, 100 Mbit/s, no loss, 1 x BDP, The measured RTT follows the setting from 0 to 300 ms, The bottleneck caps a single flow at its rate, Random loss is injected in each direction, Moving the RTT during a flow takes effect at once and the flow survives
03 Cc	4/4	103 s	Each flow runs the sender's algorithm and together they fill the bottleneck, The site B hosts send with their algorithms too, BBR takes most of a long, shallow-buffered link from CUBIC and Reno, In a deep buffer CUBIC fills the queue while BBR keeps it short
04 Mtu	5/5	76 s	The routers' and wanem's links and the data bridges are always MTU 9000, At 1500 the largest unfragmentable packet is 1500 bytes, and TCP uses a 1448-byte MSS, At 9000 jumbo frames cross the WAN end to end and through the bottleneck, All three algorithms run at MTU 9000, The hosts' MTU survives a reboot
05 Rtt Series	3/3	613 s	CUBIC alone at 500 Mbit/s, RTT 0 to 150 ms, BBR alone at 500 Mbit/s, RTT 0 to 150 ms, Reno alone at 500 Mbit/s, RTT 0 to 150 ms

Run results_2026-10-23_09-43-11: 22 passed, 0 failed.

Results: `tests/results_<timestamp>/` (report.html, log.html, output.xml), with `tests/results_latest` and `results/latest` pointing at the newest.

21. Findings

#	finding	where
1	**Alone, all three fill the link.** On a clean path the algorithm hardly matters for throughput (93–95 Mbit/s at MTU 1500); it matters for the queue it leaves behind.	§8
2	**Together, BBR v1 dominates** CUBIC and Reno whenever the buffer is around one BDP or less: ~86 of 94 Mbit/s at 30 ms, ~82 of 92 at 100 ms with ¼ BDP. Jain fairness ≈ 0.4 — close to "one flow takes all".	§9, §11
3	**Longer RTTs narrow the gap.** At 160 ms CUBIC reaches ~30 Mbit/s against BBR's ~47 (Jain 0.84). Reno stays about half of CUBIC on long paths — CUBIC's RTT-independent growth is visible.	§10
4	**Bufferbloat is a loss-based problem.** In a 4 × BDP buffer CUBIC adds ~95 ms of queueing to a 30 ms path; BBR stays near the path RTT with zero retransmits, at the same throughput.	§11
5	**1 % random loss caps CUBIC and Reno at ~4 Mbit/s** of 100, matching the Mathis formula; BBR keeps ~93.	§12
6	**BBR can starve an established CUBIC flow** to zero for tens of seconds (retransmission timeouts), not merely out-compete it.	§14
7	**RTT changes are absorbed without breaking flows.** A 30 → 120 ms jump halves CUBIC for ~5 s; BBR's rate barely moves.	§13
8	**Jumbo frames gain ~4 %** on a single flow, mostly header overhead, with far fewer retransmits.	§15
9	**At 500 Mbit/s, alone (one TCP flow, iperf3 -t 30 -b 500M), all three reach the goodput ceiling (449–472 Mbit/s) up to 100 ms RTT**, BBR with 10–50× the retransmits of CUBIC at 10 ms and below.	§16
10	**Reno recovers slowly on long fat paths:** at 150 ms one loss put it into a 340–440 Mbit/s sawtooth (391 Mbit/s average against CUBIC's 446); recovering half a 9.4 MB BDP at one segment per RTT takes about eight minutes.	§16
11	**Jumbo frames lift all three to the ceiling at every RTT up to 150 ms** (463–494 Mbit/s), with 5–14× fewer retransmits from 10 to 60 ms and none for CUBIC and BBR at 100–150 ms; Reno gains 21 % at 150 ms.	§16
12	**The hosts' tcp_mem (sized from 217 MB of RAM) bites at 500 Mbit/s × 150 ms:** a single CUBIC flow put its sender into TCP memory pressure 145 times in 30 s. It caps all three algorithms alike on the longest paths.	§4
13	**BBR is decisively better on long fat paths with random loss** (one TCP flow, iperf3 -t 30 -b 500M, 500 Mbit/s): at 0.01 % loss 3.0× / 6.6× / 13.9× CUBIC's throughput at 10 / 100 / 150 ms; at 0.1 % loss 395 Mbit/s against CUBIC's 7 at 100 ms.	§17
14	**Without loss BBR is not faster:** on a clean path with a 1 × BDP buffer all three are within 2 %; with a ¼ × BDP buffer BBR beats Reno but CUBIC was ahead of it at 100 and 150 ms.	§17

> **How far to trust these.** One run per setting, 20–40 s each, on VMs sharing a host: the direction and the size of each effect are reproducible (the Robot suites assert them on every run), the exact Mbit/s are not — a rerun moves CUBIC's and Reno's single-digit shares by a few Mbit/s. Linux 6.12's BBR is v1; BBRv3 behaves differently in findings 2, 5 and 6, by design.

22. Lessons learned building it

- **Bridges, not UDP tunnels** for anything that measures loss (homa-lab's finding, applied from the start).
- **Turn off offloads where the qdiscs are.** With GRO/LRO on, wanem's tbf counted 64 KB packets and its packet limits meant nothing; bytes per packet through the bottleneck is the check.
- **netem's default 1000-packet limit is a hidden loss source** on long fat pipes: size it from the delay line.

- **tbf drops any frame bigger than its burst.** With the links at 9000, the original burst (R / 250 Hz = 50 KB, but a 2-MTU floor for slow rates of 3 KB) would have dropped every jumbo frame at low rates; the floors are now 2 and 4 jumbo frames.
- **Cached TCP metrics** (`tcp_no_metrics_save=0`, the default) make each run start where the previous one stopped: a CUBIC run after a lossy one starts with a tiny ssthresh.
- **Alpine is not Debian:** doas instead of sudo; busybox ping has no `-M do` (iputils-ping is in the image); iputils' error text is "Message too large", not "too long".
- **A libvirt network's MTU can't change while a VM uses it**: the jumbo bridges are redefined only with the lab stopped.
- **Headless Chrome on this host hangs on any http:// page**: the portal's screenshots load a file:// copy fed with the API answers recorded from a real run.
- **Clocks after a memory restore** are wherever they were when saved; a resumed lab sets them before verifying, or TLS and log timestamps are wrong.

23. Troubleshooting

symptom	look at
verify: RTT far from the setting	`./lab.sh wan show` — the qdiscs; `./lab.sh wan reset` restores the defaults
a flow gets ~0 Mbit/s or errors	`./lab.sh cc` (iperf3 server up?); `./lab.sh ssh <host> cat /var/log/iperf3.log`; one iperf3 server serves one test at a time
wrong algorithm on a host	`/etc/sysctl.d/90-tcpcc.conf`; `lsmod | grep bbr`; `./lab.sh rebuild <host>` re-renders the seed
jumbo pings fail	`./lab.sh mtu show`; the libvirt bridges (`ip link show virbr-tccwana`): if not 9000, stop the lab and `./lab.sh up` redefines them
routers lost their config	`./lab.sh configure` (SSH) or `./lab.sh bootstrap` (console)
after a hub resume, verify fails	`./lab.sh resumed` again (clocks, SSH); `./lab.sh down; ./lab.sh up` for a cold boot
portal shows nothing	`systemctl --user status tcp-cc-lab-webapp`; `journalctl --user -u tcp-cc-lab-webapp`

24. What is not built yet

- **BBRv3** (and BBRv2's loss/ECN response): not in mainline Linux 6.12; it would need a custom kernel in the Alpine image.
- **AQM at the bottleneck** — fq_codel or CAKE instead of tbf's tail-drop FIFO — to show bufferbloat fixed in the network rather than by the sender.
- **ECN** marking (with an AQM) and DCTCP / L4S-style senders.
- **Several flows per algorithm** (iperf3 `-P`) and RTT unfairness between flows with different base RTTs (per-host netem).
- **Jitter and reordering** on the emulated link (netem supports both; reordering disturbs the loss detectors in interesting ways).
- **Packet captures** from the portal (tcpdump on wanem, a cwnd-over-time plot from `ss -ti`).

Appendix A — lab.sh

```
usage: lab.sh <command> [node...]
  image                create images/tcpcc-alpine.qcow2 (Alpine + iperf3, iproute2, ethtool, tcpdump), ~2 min
  up [node..]          create (if needed) and start the VMs (default: all); first boot of a router pushes its
day-0 config
  wait [node..]        until SSH answers (and cloud-init is done)
  down [node..]        shut the VMs down
  status | inventory   nodes, addresses, algorithms | the lab as JSON
  mtu show | set 1500 9000   the hosts' MTU (all six; r1 / r2 / wanem links always 9000)
  wan show|stats|apply RTT_MS RATE_MBIT LOSS_PCT BUFFER_BDP|reset   the WAN emulator (defaults: 30 ms RTT, 100
Mbit/s)
  cc                   each host's congestion control, qdisc, iperf3 server
  flows <sub> [opts]   compete | solo | rtt | wan  (tools/cclab.py --help)
  verify               algorithms, WAN settings, routes, ping RTT between every site A / site B pair
  configure [router..] re-apply the routers' config over SSH;  bootstrap [router..]  day-0 over the console
again
  exec <node|hosts|all> <cmd>   run a command (hosts/all: in parallel)
  test [robot args]    Robot Framework suites, tests/results_<timestamp>/
  webapp               the portal on port 8112 (user unit tcp-cc-lab-webapp)
  ssh <node> [cmd] | console <node> | log <node> [n]
  resumed [node..]     after the Lab Hub's fast resume: wait for SSH, set the clocks, verify
  rebuild [node..]     re-define domains and seeds (keeps disks; stop the nodes first)
  clean [node..]       destroy VMs and delete their disks (base images kept)
nodes: r1 r2 wanem a-cubic a-bbr a-reno b-cubic b-bbr b-reno   (libvirt domains tcpcc-<node>; hosts/wanem lab
/ lab, routers vyos / vyos, your SSH key installed)
```

`tools/cclab.py` (also `./lab.sh flows`):

```
"""tcp-cc-lab measurements: TCP flows from the hosts of one site to their peers across the emulated WAN, one
host per
congestion control algorithm (CUBIC / BBR / Reno), and the WAN emulator's settings. Stdlib only (ssh to the
VMs).

    cclab.py compete [--duration 30] [--direction a2b|b2a] [--algos cubic,bbr,reno] [--stagger 0]
                     all flows at once through the shared bottleneck: who gets what
    cclab.py solo    [--duration 20] [--direction a2b|b2a] [--algos ...]   one algorithm at a time, the link
to itself
    cclab.py sweep   [--rtts 0,10,30,60,100,150] [--rate 500] [--duration 20] [--bitrate 500M] [--direction
a2b|b2a] [--algos ...]
                     each algorithm alone at each RTT, the bottleneck at --rate; the WAN goes back to its
settings after
    cclab.py rtt     [--count 10]      ping RTT a-cubic -> b-cubic (the emulated RTT + the VMs' own ~1 ms)
    cclab.py wan     [--rtt MS] [--rate MBIT] [--loss PCT] [--buffer BDP] [--reset]   show / change the WAN
emulator
    cclab.py mtu     [--set 1500|9000]   the hosts' MTU (all six), and r1 / r2 / wanem's links (always 9000)
```

Appendix B — the WAN emulator script

`tools/wanem.sh`, installed in wanem as `/usr/local/bin/wanem`:

```
#!/bin/sh
# wanem — the WAN emulator's control (installed in the wanem VM as /usr/local/bin/wanem, run as root).
# br0 bridges eth1 (towards r1, site A) and eth2 (towards r2, site B). Each egress direction gets the same
impairment:
#   root  1:  netem  delay = RTT/2, loss = LOSS% (random, per direction), limit sized to hold a full delay
line
#   child 10: tbf    rate = the bottleneck, limit = BUFFER x BDP bytes (the bottleneck queue, where the drops
happen)
# Delay first, then the bottleneck queue: the queue a congestion control fills is tbf's, and its size does not
move
# when the delay does. "apply" changes the qdiscs in place (tc qdisc change), so running flows see the new RTT
at once.
#   wanem apply RTT_MS RATE_MBIT LOSS_PCT BUFFER_BDP | reset | show | stats (JSON) | offloads
set -eu
CONF=/etc/wanem.conf; STATE=/run/wanem.state; DEVS="eth1 eth2"
[ -r "$CONF" ] && . "$CONF"

offloads() {   # the qdiscs must see wire-size packets: no GRO / LRO (virtio guest TSO) coalescing on the way
in, no TSO/GSO out
  for d in $DEVS; do ethtool -K "$d" gro off lro off tso off gso off sg off rx-gro-hw off 2>/dev/null ||
ethtool -K "$d" gro off tso off gso off 2>/dev/null || true; done
}

apply() {
  rtt="$1"; rate="$2"; loss="$3"; buf="$4"
  # one-way delay in µs; netem queue limit = packets in flight on the delay line at the bottleneck rate, x2, +
1000;
  # tbf burst >= rate / HZ (250 or 1000) and >= 2 jumbo frames (the links are MTU 9000: a frame bigger than
the burst is
  # dropped by tbf); tbf limit = BUFFER x BDP bytes, at least 4 jumbo frames
  eval "$(awk -v r="$rtt" -v m="$rate" -v b="$buf" 'BEGIN {
    ow = r * 500; bps = m * 1e6; bdp = bps / 8 * r / 1000;
    lim = int(bps / 12000 * ow / 1e6 * 2) + 1000;
    burst = int(bps / 8 / 250); if (burst < 18028) burst = 18028;
    q = int(b * bdp); if (q < 36056) q = 36056;
    printf "OW=%d NLIM=%d BURST=%d QLIM=%d BDP=%d\n", ow, lim, burst, q, bdp }')"
  lossarg=""; [ "$(awk -v l="$loss" 'BEGIN{print (l > 0)}')" = 1 ] && lossarg="loss random ${loss}%"
  for d in $DEVS; do
    if tc qdisc show dev "$d" | grep -q "^qdisc netem 1: root"; then
      tc qdisc change dev "$d" root handle 1: netem delay "${OW}us" limit "$NLIM" $lossarg
      tc qdisc change dev "$d" parent 1:1 handle 10: tbf rate "${rate}mbit" burst "$BURST" limit "$QLIM"
    else
      tc qdisc replace dev "$d" root handle 1: netem delay "${OW}us" limit "$NLIM" $lossarg
      tc qdisc replace dev "$d" parent 1:1 handle 10: tbf rate "${rate}mbit" burst "$BURST" limit "$QLIM"
    fi
  done
  printf
'RTT_MS=%s\nRATE_MBIT=%s\nLOSS_PCT=%s\nBUFFER_BDP=%s\nONE_WAY_US=%s\nQUEUE_BYTES=%s\nBDP_BYTES=%s\nAPPLIED=%s\n'
\
    "$rtt" "$rate" "$loss" "$buf" "$OW" "$QLIM" "$BDP" "$(date +%s)" > "$STATE"
}

show() {
```

```
  [ -r "$STATE" ] && cat "$STATE"
  for d in $DEVS; do echo "--- $d (egress towards $( [ "$d" = eth1 ] && echo r1 || echo r2 ))"; tc -s qdisc
show dev "$d"; done
}

stats() {   # one JSON object: the settings and both directions' netem + tbf counters
  printf '{"settings": {'; [ -r "$STATE" ] && awk -F= 'NR>1{printf ", "} {printf "\"%s\": %s", $1, $2}'
"$STATE"; printf '}'
  for d in $DEVS; do printf ', "%s": ' "$d"; tc -s -j qdisc show dev "$d"; done; printf '}\n'
}

case "${1:-show}" in
  apply)    shift; [ $# -eq 4 ] || { echo "usage: wanem apply RTT_MS RATE_MBIT LOSS_PCT BUFFER_BDP" >&2; exit
2; }; offloads; apply "$@"; show | head -8 ;;
  reset)    offloads; apply "${RTT_MS:-30}" "${RATE_MBIT:-100}" "${LOSS_PCT:-0}" "${BUFFER_BDP:-1}"; show |
head -8 ;;
  show)     show ;;
  stats)    stats ;;
  offloads) offloads; for d in $DEVS; do ethtool -k "$d" | grep -E "^(generic|tcp-segmentation|large-receive)-
offload|^generic-receive"; done ;;
  *) echo "usage: wanem apply RTT_MS RATE_MBIT LOSS_PCT BUFFER_BDP | reset | show | stats | offloads" >&2;
exit 2 ;;
esac
```

Appendix C — lab.conf

```
# tcp-cc-lab: TCP congestion control over an emulated WAN. Two VyOS routers joined through a Linux WAN
emulator VM
# (wanem: a bridge whose two egress directions each carry netem delay = RTT/2 (+ loss) and a tbf bottleneck),
and three
# Alpine hosts behind each router, one per congestion control algorithm:
#
#   a-cubic ┐                                                          ┌ b-cubic
#   a-bbr   ┼ LAN A ─ eth2 r1 eth1 ── eth1 wanem eth2 ── eth1 r2 eth2 ─ LAN B ┼ b-bbr
#   a-reno  ┘  192.168.141/24        100.64.14.0/30 (bridged)        192.168.142/24 └ b-reno
LAB_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
LIBVIRT_URI="qemu:///system"

# VyOS base image (built once from the rolling ISO by ../cat8000v-ipsec/tools/vyos_install.py); r1 / r2 are
overlays on it
VYOS_IMAGE="/home/dcantor/cat8000v-ipsec/images/vyos-base.qcow2"
# Alpine base image for the hosts and wanem (tools/build_image.sh, from the Alpine NoCloud cloud image):
iperf3, iproute2
# (full tc / ss), ethtool, tcpdump, mtr; user lab / lab
ALPINE_SRC="/home/dcantor/srv6-core/images/alpine-3.22.4-nocloud.qcow2"
BASE_IMAGE="$LAB_DIR/images/tcpcc-alpine.qcow2"

# Out-of-band management network (isolated libvirt bridge; the host is the gateway). eth0 on every VM.
OOB_NET="tcpcc-oob"
OOB_GATEWAY="10.14.0.1"

# Data links: isolated libvirt bridges, not UDP tunnels (QEMU's UDP socket backend keeps the host's default
208 KB receive
# buffer and drops bursts silently — see homa-lab — and an unseen loss would skew exactly what this lab
measures).
#   tcpcc-wan-a  r1 eth1 <-> wanem eth1      tcpcc-lan-a  r1 eth2 + a-* eth1
#   tcpcc-wan-b  wanem eth2 <-> r2 eth1      tcpcc-lan-b  r2 eth2 + b-* eth1
NETS=(tcpcc-oob tcpcc-wan-a tcpcc-wan-b tcpcc-lan-a tcpcc-lan-b)
DATA_NETS=(tcpcc-wan-a tcpcc-wan-b tcpcc-lan-a tcpcc-lan-b)

# MTU. The infrastructure is jumbo, always: the four data bridges, r1 / r2 eth1 + eth2, wanem eth1 / eth2 /
br0 (and the
# virtio NICs of every VM on them advertise 9000). The hosts' eth1 switches between 1500 and 9000 ("lab.sh
mtu", the
# portal), all six together; the choice is kept on each host (/etc/tcpcc-mtu) across reboots. OOB management
stays 1500.
INFRA_MTU=9000
HOST_MTU=1500            # the hosts' default (a new host's first boot)
HOST_MTUS=(1500 9000)

ROUTERS=(r1 r2)
WANEM=wanem
HOSTS_A=(a-cubic a-bbr a-reno)
HOSTS_B=(b-cubic b-bbr b-reno)
HOSTS=("${HOSTS_A[@]}" "${HOSTS_B[@]}")
ALL_NODES=("${ROUTERS[@]}" "$WANEM" "${HOSTS[@]}")
declare -A ROLE=(         [r1]=router [r2]=router [wanem]=wanem [a-cubic]=host [a-bbr]=host [a-reno]=host [b-
cubic]=host [b-bbr]=host [b-reno]=host )
declare -A SITE=(         [r1]=a [r2]=b [wanem]=wan [a-cubic]=a [a-bbr]=a [a-reno]=a [b-cubic]=b [b-bbr]=b [b-
reno]=b )
declare -A CC=(           [a-cubic]=cubic [a-bbr]=bbr [a-reno]=reno [b-cubic]=cubic [b-bbr]=bbr [b-reno]=reno
```

```
)
declare -A PEER=(         [a-cubic]=b-cubic [a-bbr]=b-bbr [a-reno]=b-reno [b-cubic]=a-cubic [b-bbr]=a-bbr [b-
reno]=a-reno )
declare -A MGMT_IP=(      [r1]=10.14.0.11 [r2]=10.14.0.12 [wanem]=10.14.0.20
                          [a-cubic]=10.14.0.31 [a-bbr]=10.14.0.32 [a-reno]=10.14.0.33 [b-cubic]=10.14.0.41 [b-
bbr]=10.14.0.42 [b-reno]=10.14.0.43 )
declare -A DATA_IP=(      [r1]=192.168.141.1 [r2]=192.168.142.1
                          [a-cubic]=192.168.141.11 [a-bbr]=192.168.141.12 [a-reno]=192.168.141.13
                          [b-cubic]=192.168.142.11 [b-bbr]=192.168.142.12 [b-reno]=192.168.142.13 )
declare -A LAN_PREFIX=(   [a]=192.168.141.0/24 [b]=192.168.142.0/24 )
declare -A LAN_NET=(      [a]=tcpcc-lan-a [b]=tcpcc-lan-b )
declare -A WAN_IP=(       [r1]=100.64.14.1 [r2]=100.64.14.2 )
WAN_PREFIX_LEN=30
declare -A WAN_NET=(      [r1]=tcpcc-wan-a [r2]=tcpcc-wan-b )
declare -A NODE_IDX=(     [r1]=1 [r2]=2 [wanem]=3 [a-cubic]=4 [a-bbr]=5 [a-reno]=6 [b-cubic]=7 [b-bbr]=8 [b-
reno]=9 )
declare -A CONSOLE_PORT=( [r1]=5601 [r2]=5602 [wanem]=5603 [a-cubic]=5604 [a-bbr]=5605 [a-reno]=5606 [b-
cubic]=5607 [b-bbr]=5608 [b-reno]=5609 )
declare -A VCPU=(         [r1]=2 [r2]=2 [wanem]=2 [a-cubic]=1 [a-bbr]=1 [a-reno]=1 [b-cubic]=1 [b-bbr]=1 [b-
reno]=1 )
declare -A RAM_MIB=(      [r1]=1024 [r2]=1024 [wanem]=512 [a-cubic]=256 [a-bbr]=256 [a-reno]=256 [b-cubic]=256
[b-bbr]=256 [b-reno]=256 )
BUILDER_CONSOLE=5610
MAC_OUI="52:54:00:ce"     # MACs 52:54:00:ce:<idx>:<nic> (hex); :c6-:cd taken by other labs

# WAN emulator defaults, applied at wanem's boot and by "lab.sh wan reset": round-trip time (split evenly
between the two
# directions), the bottleneck rate, random loss per direction, and the bottleneck buffer in multiples of the
BDP (rate x RTT)
WAN_RTT_MS=30
WAN_RATE_MBIT=100
WAN_LOSS_PCT=0
WAN_BUFFER_BDP=1
WAN_RTT_MAX_MS=300

# Host TCP settings: the per-host algorithm (CC above), fq on eth1 (pacing; BBR relies on it), buffers large
enough that
# the window never caps a flow at 300 ms x 100 Mbit/s (3.75 MB), and no cached metrics (a previous run's
ssthresh would
# otherwise start the next one), iperf3 servers on 5201
HOST_SYSCTL=( "net.core.default_qdisc=fq" "net.core.rmem_max=33554432" "net.core.wmem_max=33554432"
              "net.ipv4.tcp_rmem=4096 131072 33554432" "net.ipv4.tcp_wmem=4096 65536 33554432"
"net.ipv4.tcp_no_metrics_save=1" )
```

Appendix D — every TCP setting on the hosts

`sysctl -a | grep -E '^net.ipv4.tcp_|^net.core.(rmem_|wmem_|default_qdisc|netdev_max_backlog|optmem_max)'` on a-cubic; identical on all six hosts except `tcp_congestion_control`. eth1's qdisc:

```
qdisc fq 8002: root refcnt 2 limit 10000p flow_limit 100p buckets 1024 orphan_mask 1023 bands 3 priomap 1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1 weights 589824 196608 65536 quantum 3028b initial_quantum 15140b low_rate_threshold 550Kbit refill_delay 40ms timer_slack 10us horizon 10s horizon_drop
```

```
net.core.default_qdisc = fq
net.core.netdev_max_backlog = 1000
net.core.optmem_max = 131072
net.core.rmem_default = 212992
net.core.rmem_max = 33554432
net.core.wmem_default = 212992
net.core.wmem_max = 33554432
net.ipv4.tcp_abort_on_overflow = 0
net.ipv4.tcp_adv_win_scale = 1
net.ipv4.tcp_allowed_congestion_control = reno cubic bbr
net.ipv4.tcp_app_win = 31
net.ipv4.tcp_autocorking = 1
net.ipv4.tcp_available_congestion_control = reno cubic bbr
net.ipv4.tcp_available_ulp = mptcp
net.ipv4.tcp_backlog_ack_defer = 1
net.ipv4.tcp_base_mss = 1024
net.ipv4.tcp_challenge_ack_limit = 2147483647
net.ipv4.tcp_child_ehash_entries = 0
net.ipv4.tcp_comp_sack_delay_ns = 1000000
net.ipv4.tcp_comp_sack_nr = 44
net.ipv4.tcp_comp_sack_slack_ns = 100000
net.ipv4.tcp_congestion_control = cubic
net.ipv4.tcp_dsack = 1
net.ipv4.tcp_early_demux = 1
net.ipv4.tcp_early_retrans = 3
net.ipv4.tcp_ecn = 2
net.ipv4.tcp_ecn_fallback = 1
net.ipv4.tcp_ehash_entries = 2048
net.ipv4.tcp_fack = 0
net.ipv4.tcp_fastopen = 1
net.ipv4.tcp_fastopen_blackhole_timeout_sec = 0
net.ipv4.tcp_fin_timeout = 60
net.ipv4.tcp_frto = 2
net.ipv4.tcp_fwmark_accept = 0
net.ipv4.tcp_invalid_ratelimit = 500
net.ipv4.tcp_keepalive_intvl = 75
net.ipv4.tcp_keepalive_probes = 9
net.ipv4.tcp_keepalive_time = 7200
net.ipv4.tcp_l3mdev_accept = 0
net.ipv4.tcp_limit_output_bytes = 1048576
net.ipv4.tcp_low_latency = 0
net.ipv4.tcp_max_orphans = 1024
net.ipv4.tcp_max_reordering = 300
net.ipv4.tcp_max_syn_backlog = 128
net.ipv4.tcp_max_tw_buckets = 1024
net.ipv4.tcp_mem = 2418 3227 4836
net.ipv4.tcp_migrate_req = 0
net.ipv4.tcp_min_rtt_wlen = 300
net.ipv4.tcp_min_snd_mss = 48
net.ipv4.tcp_min_tso_segs = 2
net.ipv4.tcp_moderate_rcvbuf = 1
net.ipv4.tcp_mtu_probe_floor = 48
net.ipv4.tcp_mtu_probing = 0
net.ipv4.tcp_no_metrics_save = 1
net.ipv4.tcp_no_ssthresh_metrics_save = 1
net.ipv4.tcp_notsent_lowat = 4294967295
net.ipv4.tcp_orphan_retries = 0
net.ipv4.tcp_pacing_ca_ratio = 120
net.ipv4.tcp_pacing_ss_ratio = 200
net.ipv4.tcp_pingpong_thresh = 1
net.ipv4.tcp_plb_cong_thresh = 128
net.ipv4.tcp_plb_enabled = 0
net.ipv4.tcp_plb_idle_rehash_rounds = 3
net.ipv4.tcp_plb_rehash_rounds = 12
net.ipv4.tcp_plb_suspend_rto_sec = 60
net.ipv4.tcp_probe_interval = 600
net.ipv4.tcp_probe_threshold = 8
net.ipv4.tcp_recovery = 1
net.ipv4.tcp_reflect_tos = 0
net.ipv4.tcp_reordering = 3
net.ipv4.tcp_retrans_collapse = 1
net.ipv4.tcp_retries1 = 3
net.ipv4.tcp_retries2 = 15
net.ipv4.tcp_rfc1337 = 1
net.ipv4.tcp_rmem = 4096 131072 33554432
net.ipv4.tcp_rto_min_us = 200000
net.ipv4.tcp_sack = 1
net.ipv4.tcp_shrink_window = 0
net.ipv4.tcp_slow_start_after_idle = 1
net.ipv4.tcp_stdurg = 0
net.ipv4.tcp_syn_linear_timeouts = 4
net.ipv4.tcp_syn_retries = 6
net.ipv4.tcp_synack_retries = 5
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_thin_linear_timeouts = 0
net.ipv4.tcp_timestamps = 1
net.ipv4.tcp_tso_rtt_log = 9
net.ipv4.tcp_tso_win_divisor = 3
net.ipv4.tcp_tw_reuse = 2
net.ipv4.tcp_window_scaling = 1
net.ipv4.tcp_wmem = 4096 65536 33554432
net.ipv4.tcp_workaround_signed_windows = 0
```